

EE351

微机原理与微系统

姜俊敏

电子与电子工程系

第五章 数据通信和总线

本章学习目标

5.1 通信的有关概念

5.2 串行接口

5.3 并行总线接口的使用方法

1 通信的有关概念

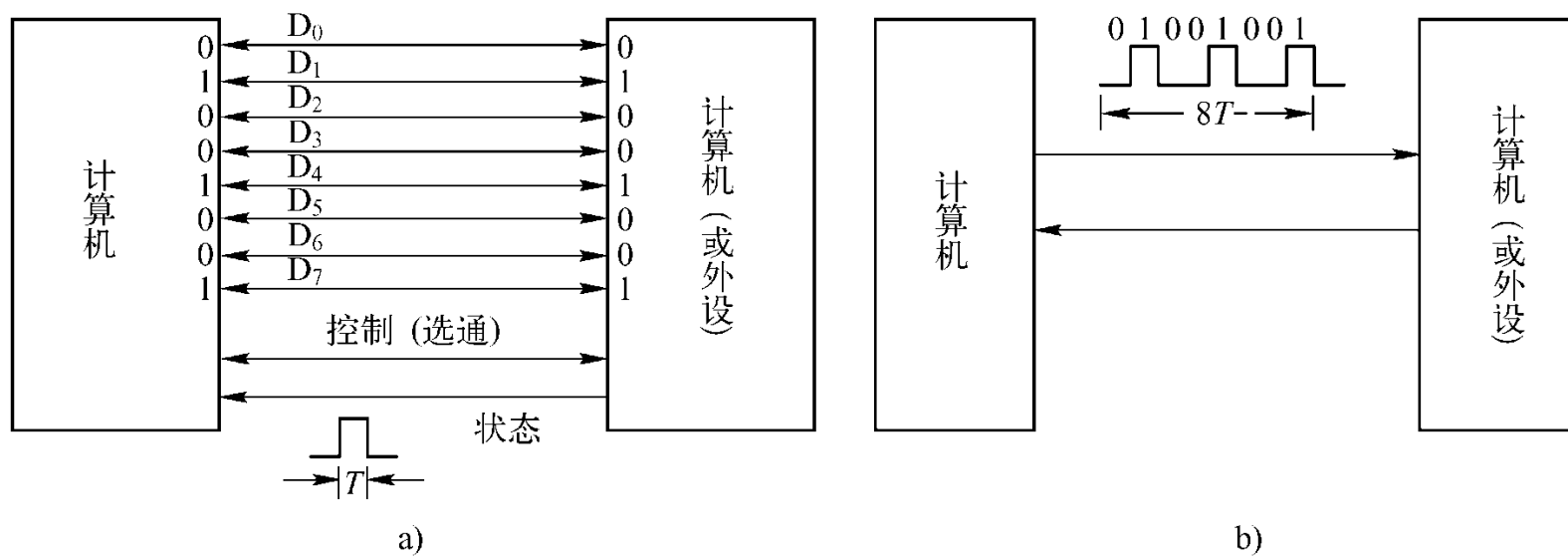
通信： CPU与外部设备之间, 及计算机之间的信息交换。

通信分类： 并行通信和串行通信

◇ **并行通信：** 以字节(Byte)或字节的倍数为传输单位;

◆ 一次传送一个(以上)字节的数据, 数据各位同时传送;

◇ **串行通信：** 用2(或3)根数据信号线相连, 数据在一根数据信号线上一位一位地顺序传送, 每位数据都占据一个固定的时间长度。



并行通信和串行通信的连接示意图

1. 通信的有关概念

并行通信

- 适合于外部设备与微机之间进行**近距离**、**大量**和**快速**的信息交换。**计算机内部的各个总线传输数据**时就是以并行方式进行的。
- **并行通信的特点**就是传输速度快，但当距离较远、位数较多时，通信线路复杂且成本高。

◇ 串行通信

- ◆ 与并行通信相比，串行通信的**优点**是**传输线少**、成本低、**适合远距离传送**及易于扩展。**缺点**是**速度慢**、传输时间长等。
- ◆ 如计算机上常用的**COM口**设备、**USB设备**和**网络通信**等设备都采用串行通信。

1 通信的有关概念



DB9型COM串行口(RS232公口)

LPT接口

USB口

DB25型LPT并行
打印机接口

1 串行通信的相关概念

1、 串行通信的分类

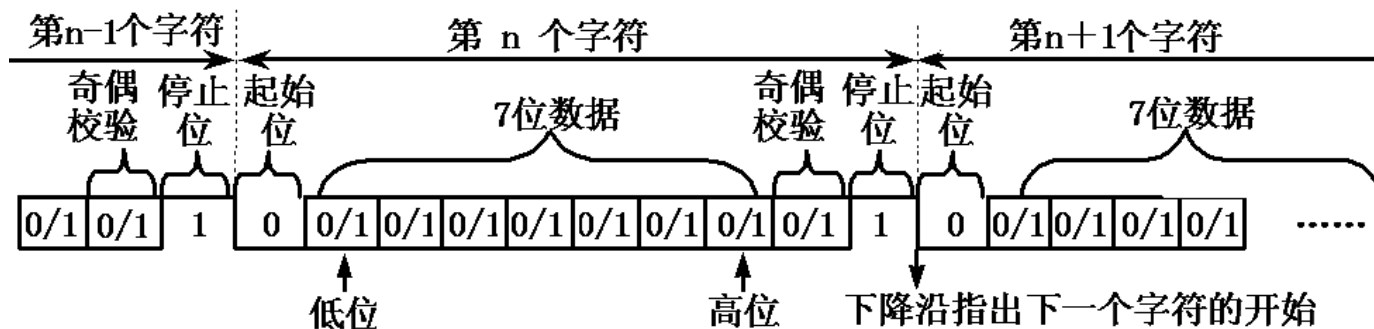
(1) 按照串行数据的同步方式分类

串行通信可以分为：同步通信和异步通信两类。

①异步通信

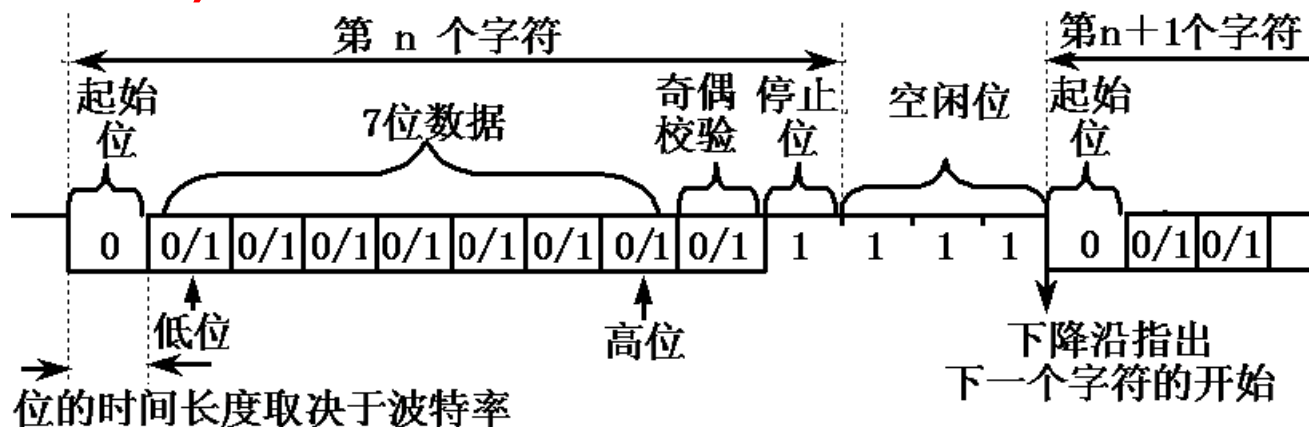
- ◇ 异步通信(Asynchronous Communication)方式, 接收器和发送器使用各自的时钟, 它们的工作是非同步的。
- ◇ 数据传送以字符为单位, 字符与字符间的传送是完全异步的, 位与位之间的传送基本上是同步的。
- ◇ 异步传送中,每个字符要用起始位和停止位作为字符开始和结束的标志, 以字符为单位逐个发送和接收。

典型的异步通信格式如图所示。



a) 数据字为7位ASCII码时的通信格式

a) 数据字为7位ASCII码时的通信格式



b) 有空闲位时的通信格式

异步通信的格式 b) 有空闲位时的通信格式

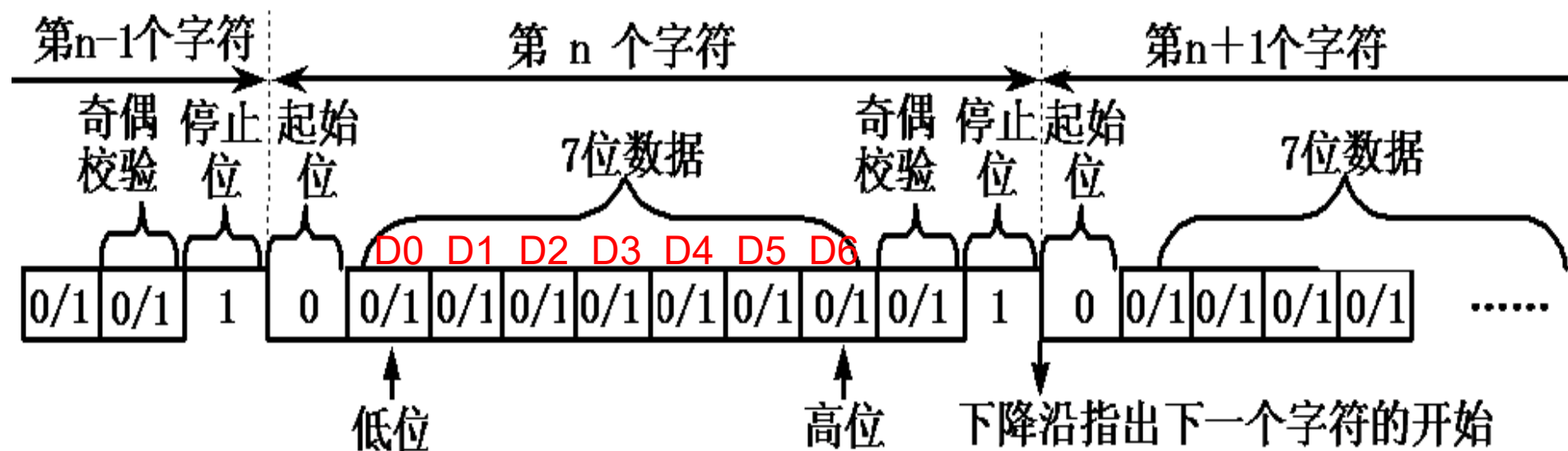
① 异步通信

异步传送时，每个字符的组成格式

- 首先用一个起始位0表示字符的开始；
- 后面紧跟着的是字符的数据字，数据字通常是7位或8位数据(低位在前,高位在后: D0D1D2D3D4D5D6D7)，在数据字中可根据需要加入奇偶校验位；
- 最后是停止位1，其长度可以是一位或两位。串行传送的数据字加上成帧信号的起始位和停止位就形成了一个串行传送的帧(字符帧)。
- 起始位用逻辑“0”低电平表示，停止位用逻辑“1”高电平表示。

① 异步通信

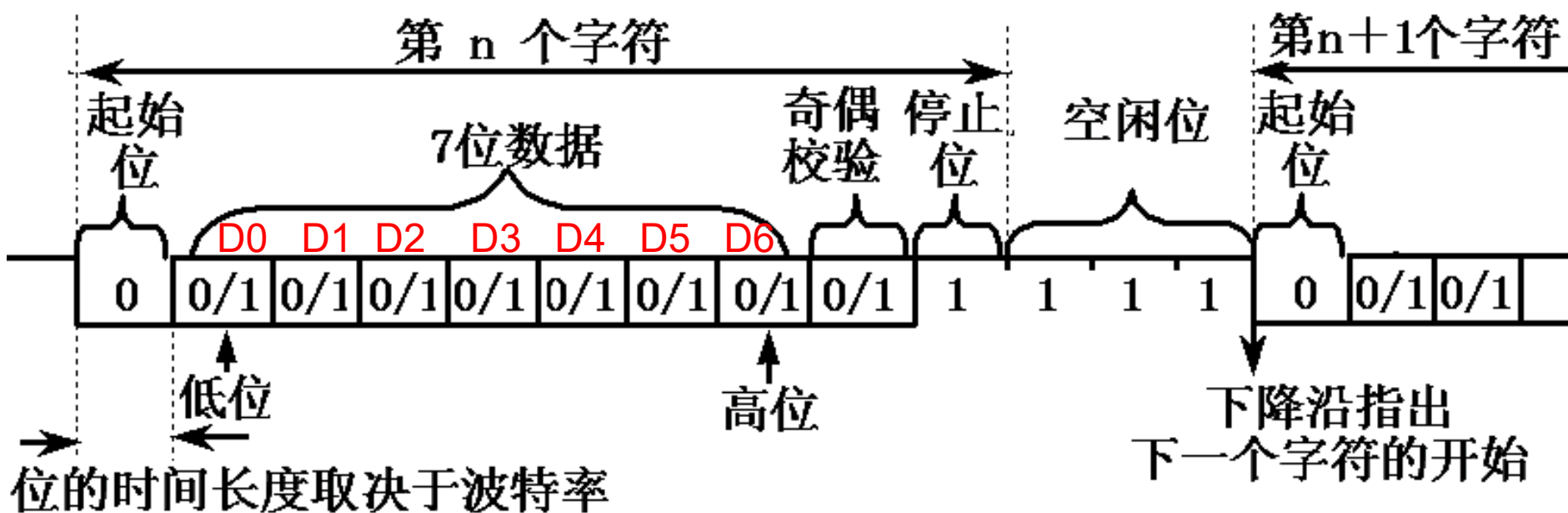
图a所示为数据字为7位的ASCII码，第8位是奇偶校验位，加上起始位、停止位，一个字符帧由10位组成。形成帧信号后，字符便一个一个地进行传送。



a) 数据字为7位ASCII码时的通信格式

① 异步通信

在异步传送中，字符间隔不固定，在停止位后可以加空闲位，空闲位用高电平表示，用于等待发送。这样，接收和发送可以随时进行，不受时间的限制。



b) 有空闲位时的通信格式

① 异步通信

在异步数据传送中, 通信双方必须约定好两项事宜:

- 字符格式: 包括字符的编码形式、奇偶校验以及起始位和停止位的规定。
- 通信速率: 通信速率通常使用比特率来表示。
 - 比特率是数字信号的传输速率, 它用单位时间内传输的二进制代码的有效位(bit)数来表示;
 - 其单位为每秒比特数bit/s (或bps)、每秒千比特数(Kbps)或每秒兆比特数(Mbps)来表示。

bps: bit per second

①异步通信

波特率与比特率

- 波特率指数据信号对载波的调制速率，它用单位时间内载波调制状态改变次数来表示，其单位为波特(Baud)。
- 波特率与比特率的关系是比特率=波特率×单个调制状态对应的二进制位数。
- 在信息传输通道中，携带数据信息的信号单元叫码元，每秒钟通过信道传输的码元数称为码元传输速率，简称波特率。波特率是传输通道频宽(带宽)的指标。

(1) 按照串行数据的同步方式分类 ① 异步通信

波特率与比特率：

(百度百科对波特率解释:)波特率是单片机或计算机在串口通信时的速率。
数据传送速率用字符(帧)/秒表示时可理解为波特率。

例如，数据传送速率为120字符(帧)/秒(这个速率可称为波特率, 即120Baud)。

而每一字符帧为10位(1个起始位,1个停止位,8个数据位), 则其传送的比特率为 $10 \times 120 = 1200 \text{ bit/s} = 1200\text{bps}$ 。

在后面的描述中，为了适应习惯用法，将比特率和波特率统一使用波特率来表示。

(1) 按照串行数据的同步方式分类

② 同步通信

同步通信(Synchronous Communication)是一种连续串行传送数据的通信方式,一次通信只传送一帧信息。

这里的**信息帧(数据块)**和异步通信中的**字符帧**不同,通常含有若干个数据字符。

即**数据传送**是以**数据块(一组字符)**为单位,字符与字符之间、字符内部的位与位之间都**同步**,无间隔。

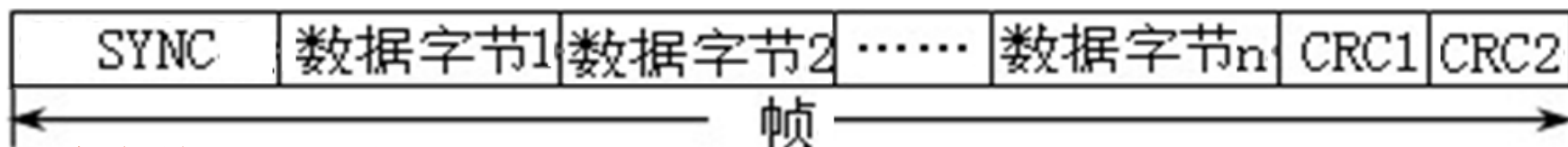
接收时钟与发送时钟严格同步,通常要有同步时钟。

根据控制规程,数据格式分为**面向字符**及**面向比特**两种。

②同步通信

(a) 面向字符型的数据格式

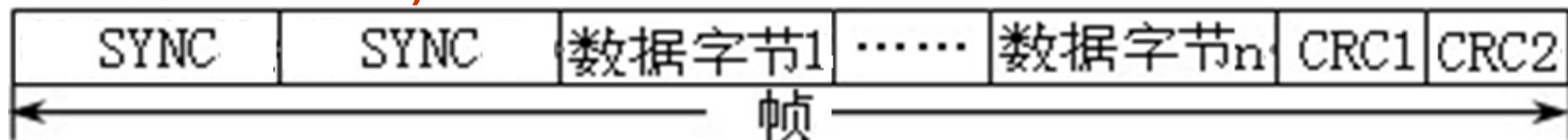
面向字符型的同步通信数据格式可采用单同步、双同步和外同步三种数据格式，如图所示。



单同步字符**SYNC** (如
ASCII码的**SYN**码**16H**)

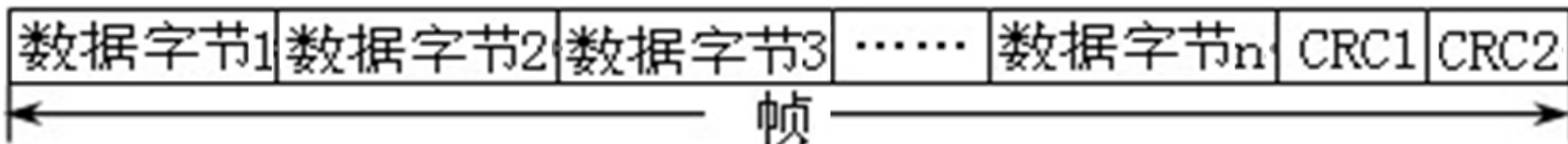
(a) 单同步字符帧结构

校验字符**CRC**



双同步字符**SYNC** (如
国际通用标准**EB90H**)

(b) 双同步字符帧结构



用一条**专用控制线**
来传送同步字符

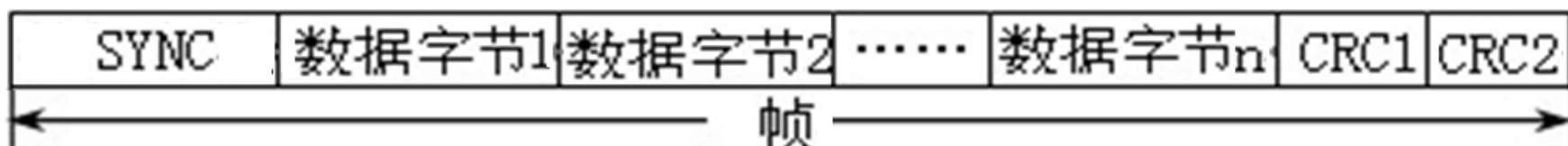
(c) 外同步字符帧格式

面向字符型同步通信数据格式

②同步通信 (a) 面向字符型的数据格式

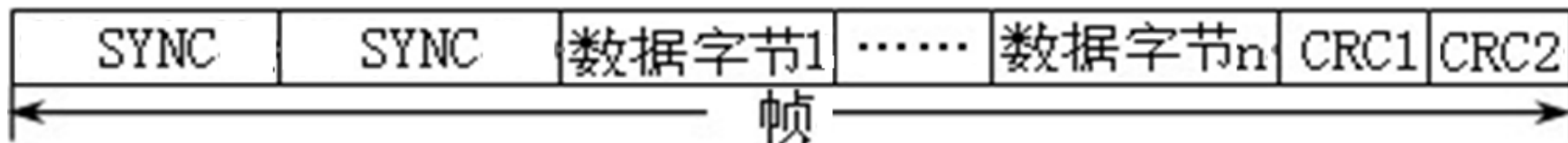
单同步、双同步

- 单同步和双同步均由同步字符、数据字符和校验字符CRC (Cyclic Redundancy Check) 等三部分组成。



(a) 单同步字符帧结构

- 单同步是指在传送数据之前先传送一个同步字符“SYNC”，双同步则先传送两个同步字符“SYNC”。



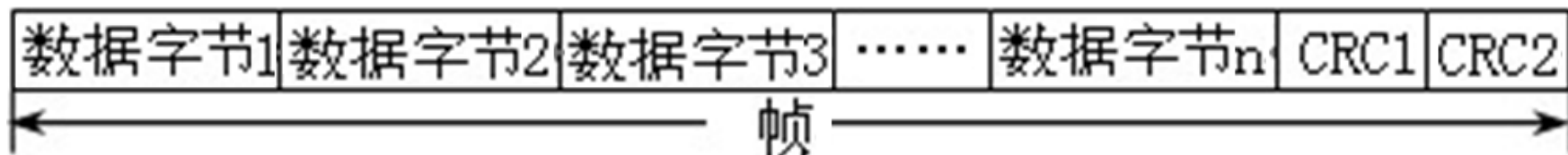
(b) 双同步字符帧结构

同步字符可采用统一标准格式,也可用户约定,单同步字符SYNC可采用如ASCII码规定的SYN码16H; 双同步字符SYNC一般采用国际通用标准代码EB90H。

②同步通信 (a) 面向字符型的数据格式

外同步

- 外同步通信的数据格式中没有同步字符，而是用一条**专用控制线**来传送**同步字符**，使接收端及发送端实现同步。当每一帧信息结束时均用两个字节的循环控制码CRC为结束。



(c) 外同步字符帧格式

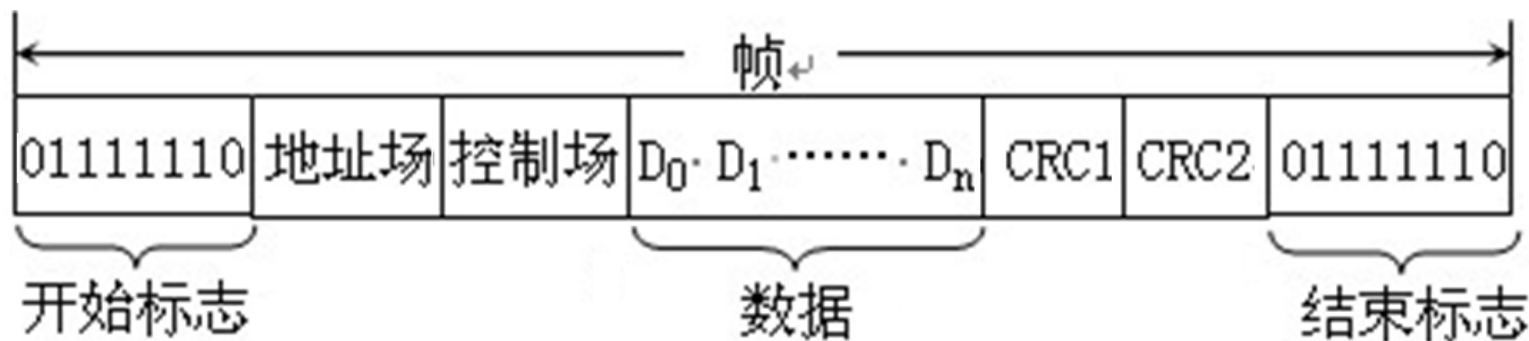
②同步通信 (b) 面向比特型的数据格式

根据同步数据链路控制规程(SDLC), 面向比特型的数据每帧由六个部分组成。

- 第一部分是开始标志“7EH (0111 1110B)”;
- 第二部分是一个字节的地址场;
- 第三部分是一个字节的控制场;
- 第四部分是需传送的数据, 数据都是位(bit)的集合;
- 第五部分是两个字节的循环控制码CRC;
- 最后部分又是“7EH (0111 1110B)”, 作为结束标志。

②同步通信 (b) 面向比特型的数据格式

面向比特型的数据格式如图所示。



面向比特型同步通信数据格式

- ◇ 注意：在SDLC规程中不允许在数据段和CRC段中出现六个“1”，否则会误认为是结束标志。
- ◇ 要求在发送端进行检验，当连续出现五个“1”时，则立即插入一个“0”，到接收端要将这个插入的“0”去掉，恢复原来的数据，保证通信的正常进行。

(1) 按照串行数据的同步方式分类

②同步通信

同步通信优缺点

- 数据传输速率较高，通常可达56000bps或更高，适用于传送信息量大、传送速率高的系统中，
- 缺点是要求发送时钟和接收时钟保持严格同步，故发送时钟除应和发送波特率保持一致外，还要求把它同时传送到接收端去。

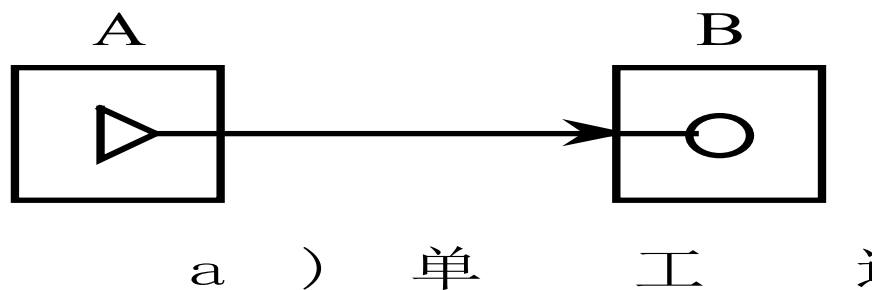
1、串行通信的分类

(2) 按照数据的传送方向分类

按照数据传送方向，串行通信可分为单工、半双工和全双工三种方式。

◇ 图a为单工通信方式(Simplex)。A为发送站，B为接收站，数据只能由A发至B，而不能由B传送到A。

单工通信类似无线电广播，电台发送信号，收音机接收信号，收音机永远不能发送信号。



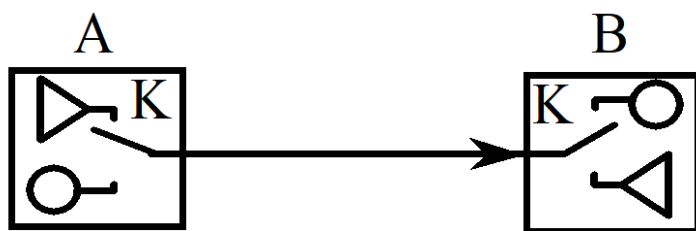
a) 单工通信方式
点-点通信方式

(2) 按照数据的传送方向分类

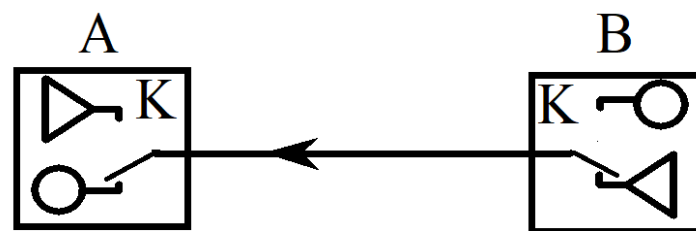
图b为半双工通信方式(Half Duplex)。数据可从A发送到B，也可由B发送到A。

不过，由于使用一根线连接，发送和接收不可能同时进行，同一时间只能作一个方向的传送，其传送方向由收发控制开关K来控制。

半双工通信方式类似对讲机，某时刻A发送B接收，另一时刻B发送A接收，双方不能同时进行发送和接收。



半双工通信方式



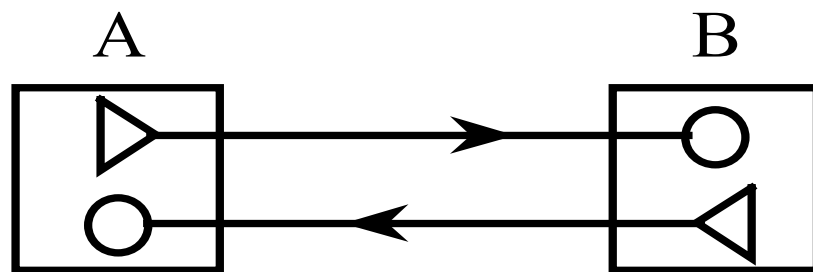
半双工通信方式

b) 半双工通信方式
点-点通信方式

(2) 按照数据的传送方向分类

图c为全双工通信方式(Full Duplex)。在这种方式中，分别用2根独立的传输线来连接发送方和接收方，A、B既可同时发送，又可同时接收。

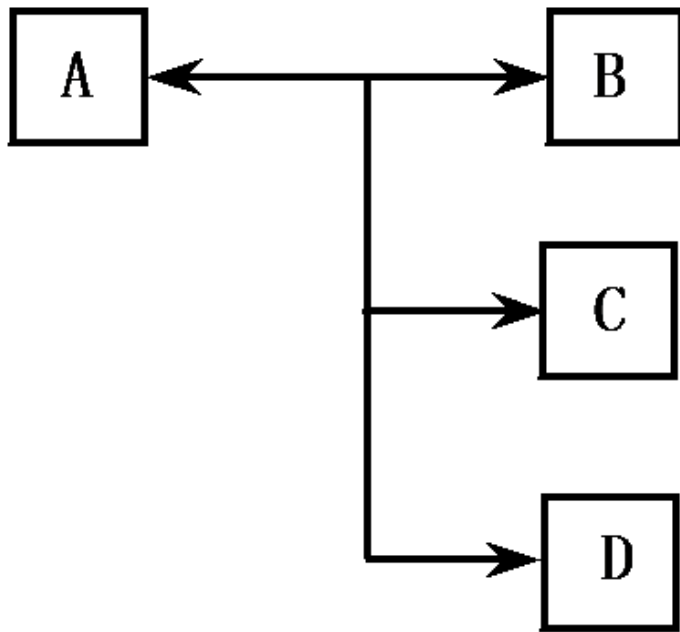
全双工通信工方式类似电话机，双方可以同时进行数据的发送和接收。



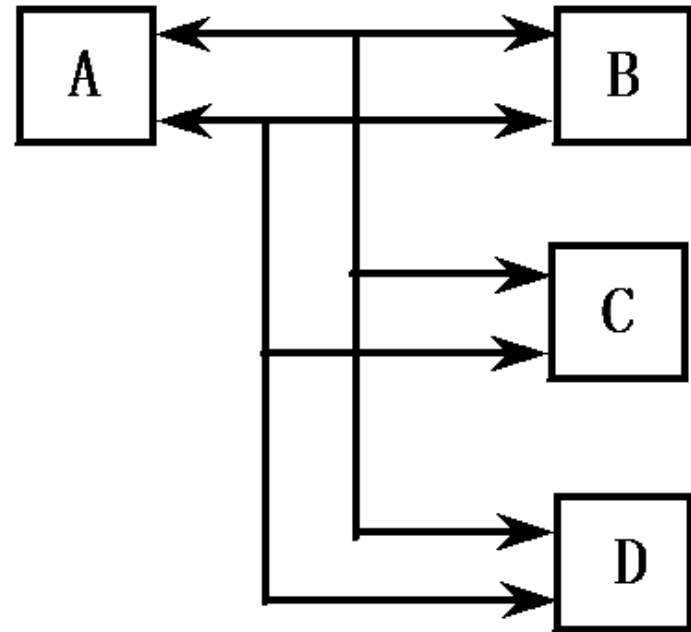
c) 全双工通信方式
图8-5 点-点通信方式

(2) 按照数据的传送方向分类

图示为主从多终端通信方式。A可以向多个终端(B, C, D...)发出信息。在A允许的条件下，可以控制管理B, C, D等在不同的时间向A发出信息。又分为多终端半双工通信和多终端全双工通信。



a) 多终端半双工通信方式



b) 多终端全双工通信方式

主从多终端通信方式

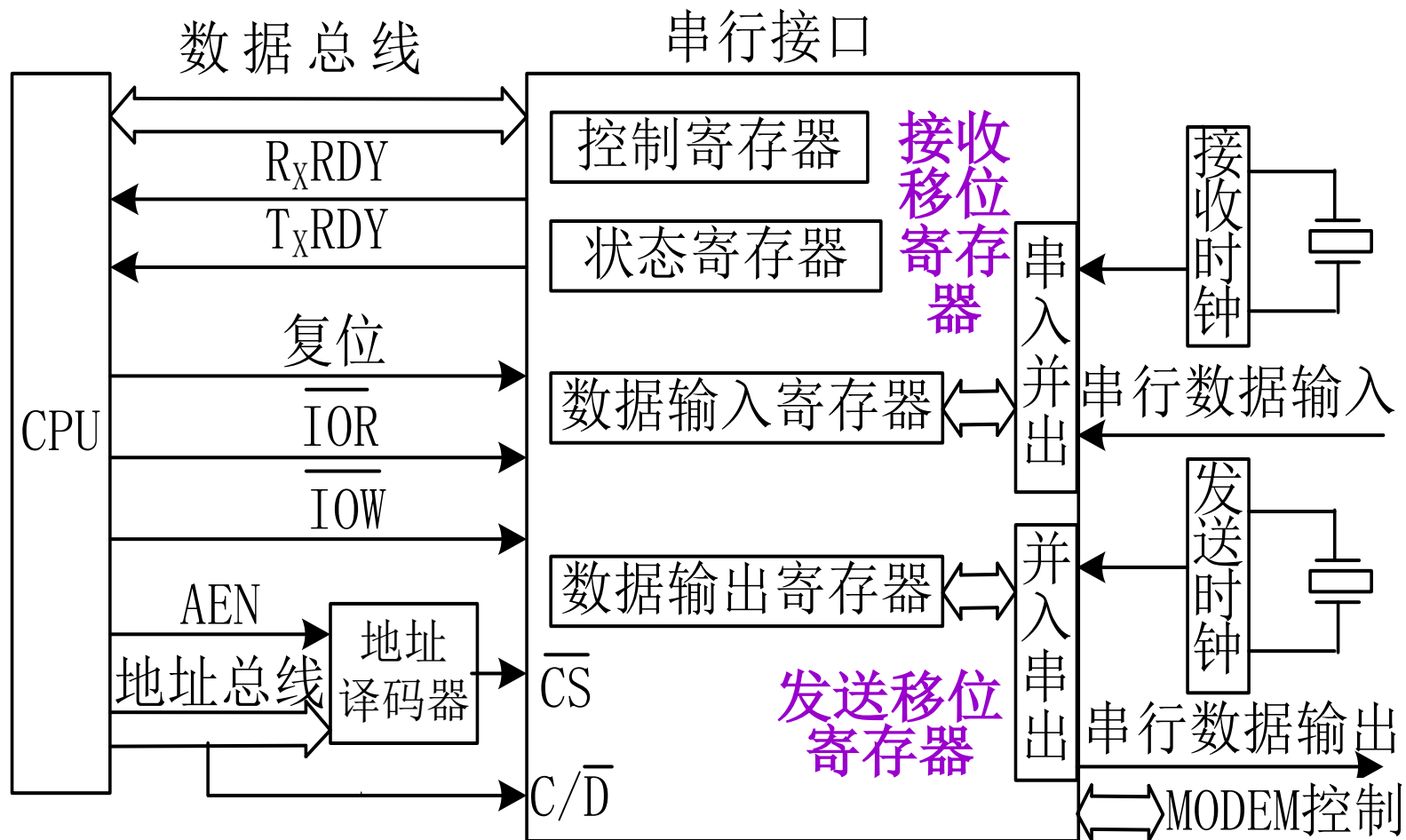
1 串行通信的相关概念

2、串行接口

作用：串行通信中的数据是一位一位依次传送的，而计算机中数据是并行传送的。因此，发送端必须把并行数据变成串行才能传送，接收端接收到的串行数据又需要变换成并行数据才可以送给计算机。上述并→串或串→并的转换既可以用软件实现，也可用硬件实现。由于用软件实现会使CPU的负担增加，目前往往用硬件（串行接口）完成这种转换。

2、串行接口

串行接口通过系统总线和CPU相连，如图所示。



2、串行接口

串行接口主要由4部分组成

- **数据输入寄存器**。在输入过程中，串行数据一位一位地从传输线进入串行接口的**接收移位寄存器**，经过串入并出电路的转换，当接收完一个字符之后，数据就从接收移位寄存器传送到**数据输入缓冲器**，等待CPU读取。
- **数据输出寄存器**。当CPU输出数据时，先送到**数据输出缓冲器**，然后，数据由输出寄存器传到**发送移位寄存器**，经过并入串出电路转换一位一位地通过输出传输线送到外设。

2、串行接口

串行接口主要由4部分组成

- **状态寄存器**。状态寄存器用来存放外设运行的状态信息，CPU通过访问这个寄存器来了解某个外设的状态，进而控制外设的工作，以便与外设进行数据交换。
- **控制寄存器**。 串行接口中有一个控制寄存器，CPU对外设设置的工作方式命令、操作命令都存放在控制寄存器中，通过控制寄存器控制外设运行。

2、串行接口

串行接口基本工作原理

- 串行发送时，CPU通过数据总线把8位并行数据送到数据输出寄存器，然后送给并行输入/串行输出移位寄存器，并在发送时钟和发送控制电路控制下通过串行数据输出端一位一位串行发送出去。
- 起始位和停止位是由串行接口在发送时自动添加上去的。串行接口发送完一帧后产生中断请求，CPU响应后可以把下一个字符送到发送数据缓冲器。

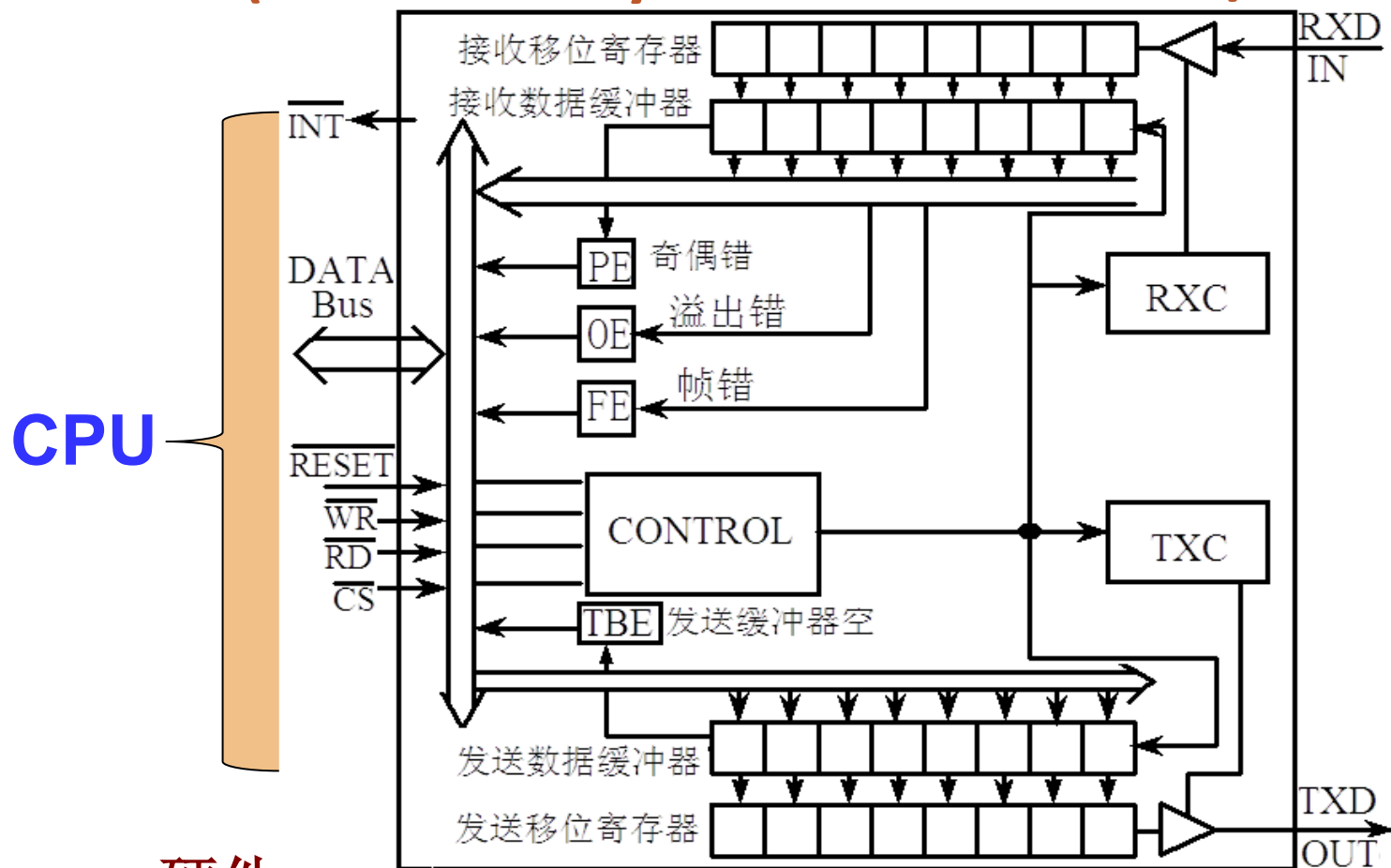
2、串行接口

串行接口基本工作原理

- 串行接收时，串行接口监视串行数据输入端，并在检测到有一个低电平（起始位）时就开始一个新的字符接收过程。
- 串行接口每接收到一位二进制数据位后就使接收移位寄存器(即串行输入-并行输出寄存器)左移一次;
- 连续接收到一个字符(字节)后将其并行传送到数据输入寄存器，并产生中断促使CPU从中取走所接收的字符。

2、串行接口 一常见芯片: 通用异步接收器/发送器UART

◇ UART(Universal Asynchronous Receiver/Transmitter)



硬件
UART的结构

2、串行接口

UART中3种出错标志:

奇偶错误, 帧错误, 溢出(丢失)错误。

奇偶错误 (Parity error)。为了检测传送中可能发生的错误, UART在发送时会检查每个要传送的字符中的“1”的个数, 自动在奇偶校验位上添加“1”或“0”, 使得“1”的总和 (包括奇偶校验位) 在偶校验时为偶数, 奇校验时为奇数。

- UART在接收时会检查字符中的每一位(包括奇偶校验位), 计算其“1”的总和是否符合奇偶检验的要求, 以确定是否发生传送错误。

2、串行接口

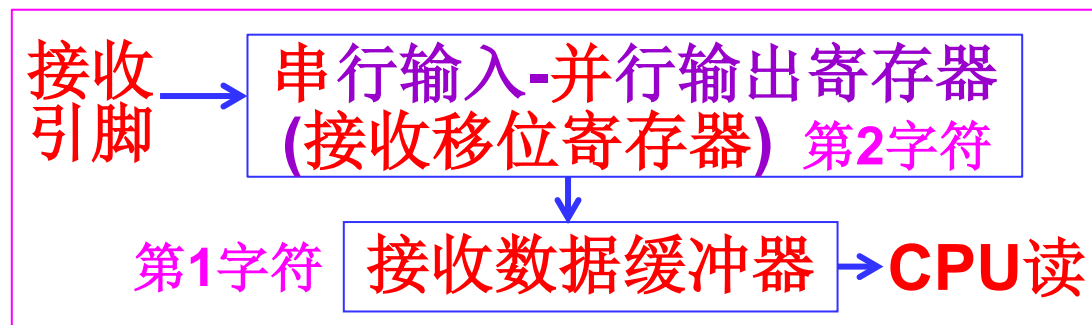
UART中3种出错标志:

- 帧错误（**Frame error**），表示**字符格式不符合规定**。
- 虽然接收端和发送端的时钟没有直接的联系，但是因为**接收端总是在每个字符的起始位处进行一次重新定位**，因此，必须要保证每次采样都对应一个数据位。
- 如果**接收时钟和发送时钟的频率相差太大**，引起在起始位之后刚采样几次就造成错位时，会出现采样造成的接收错误。
- 如果遇到这种情况，就会出现**停止位(按规定应为高电平)为低电平** (此情况下,未必每个停止位都是低电平), 从而引起**信息帧格式错误**, 帧错误标志**FE置位**。

接收	0	1	0	0	0	1	0	0	0	0	停止位
发送	0	1	0	0	1	0	0	0	0	0	1

起始位停止位

2、串行接口



UART3种出错标志:

- 溢出(丢失)错误 (Overrun error)。UART是一种双缓冲器结构。UART接收端在接收到第一个字符后便放入接收数据缓冲器，然后就继续从RXD线上接收第二个字符，并等待CPU从接收数据缓冲器中取走第一个字符。
- 如果CPU很忙，一直没有机会取走第一个字符，以致接收到的第二个字符进入接收数据缓冲器而造成第一个字符被丢失，于是产生了溢出错误，UART自动使溢出错误标志OE置位。

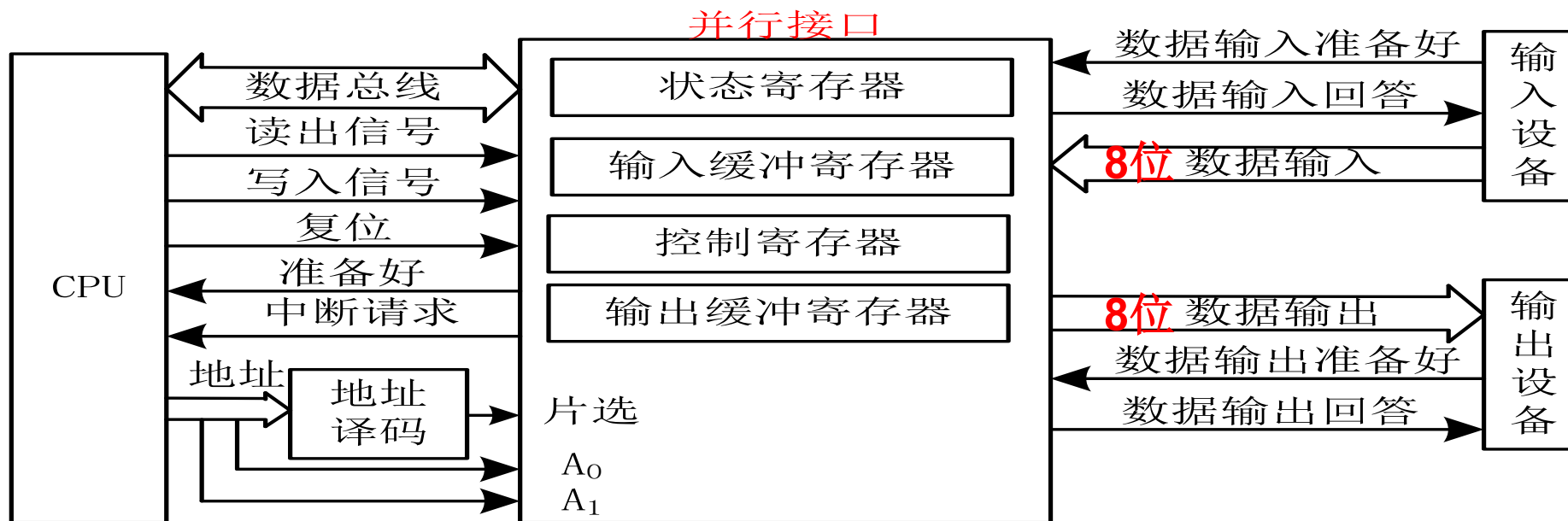
8.1.2 并行通信中的相关概念

1、并行接口

定义：实现并行通信的接口电路。

分类：输入并行接口、输出并行接口和输入/输出并行接口。

并行通信以同步方式传输,其特点是:传输速度快;硬件开销大;适合近距离传输。



1、并行接口

并行接口的传输信息包括：状态信息、控制信息、数据信息。

- 状态信息: 状态信息表示外设当前所处的工作状态。例如, 准备好信号“READY”=1表示输入接口已经准备好, 可以和CPU交换数据; 忙信号“BUSY”=1表示接口正在传输信息, CPU需要等待。
- 控制信息: 控制信息是由CPU发出的, 用于控制外设接口的工作方式以及外设的启动和复位等。
- 数据信息: CPU与并行接口交换的主要内容。

1、并行接口

典型并行接口与CPU、外设连接图如图所示。

并行接口

撤销

数据输入准备好

数据输入回答

8位 数据输入

8位 数据输出

数据输出应答

启动(选通)信号

输入设备

输出设备

状态寄存器

输入缓冲寄存器

控制寄存器

输出缓冲寄存器

片选

A₀

A₁

数据总线

读出信号

写入信号

复位

准备好

中断请求

地址

地址译码

CPU

典型并行接口电路图

2 并行通信中的相关概念

2、并行接口电路组成

- 输入缓冲寄存器。输入数据缓冲器主要功能是负责接收设备送来的数据，CPU通过读操作指令(~~IN~~) **MOVX A, @DPTR**执行读操作, 从输入数据缓冲器读取数据。
- 输出缓冲寄存器。输出数据缓冲器主要功能是负责接收CPU送来的数据，如果设备处于空闲状态，则从输出数据缓冲器取走数据，接口通知CPU进行下一次输出操作。

2 并行通信中的相关概念

2、并行接口电路组成

- 状态寄存器。状态寄存器用来存放外设运行状态信息，CPU通过访问状态寄存器来了解外设状态，进而控制外设的工作。
- 控制寄存器。并行接口中有一个控制寄存器，CPU对外设设置的工作方式命令、操作命令都存放在控制寄存器中，通过控制寄存器控制外设的运行。
- 数据信息。CPU与并行接口交换的主要内容。

2 并行通信中的相关概念

3、并行通信接口的基本输入/输出工作过程

(1) 输入过程

- 外设首先将并行传输的数据放到外设与接口之间的数据总线上，并使“数据输入准备好”状态选通信号有效，该选通信号使数据输入到接口的输入数据缓冲器内。
- 当数据写入输入数据缓冲器后，接口使“数据输入应答”信号有效(表示接口已收到外设数据)，作为对外设输入的响应。
- 外设收到此信号后，便撤销输入数据和“数据输入准备好”信号。

(1) 输入过程

- 数据到达接口后，接口在状态寄存器中设置“输入准备好”状态位为1(表示接口输入缓冲器已满), 以便CPU进行查询; 接口也可以在此时向CPU发送中断请求, 表示数据已输入到接口。
- CPU既可以用查询程序方式，也可以用程序中断方式来读取接口中的数据。
- CPU从输入缓冲器中读取数据后，接口自动清除状态寄存器中“输入准备好”状态位为0(表示接口输入缓冲器已空), 并使数据总线处于高阻状态。至此，一个数据的传送结束。

3、并行通信接口的基本输入/输出工作过程

(2) 输出过程

- 当外设从接口输出缓冲寄存器取走数据后, 接口就将状态寄存器中“输出准备好”状态位置1(表示输出缓冲器已空), 表示CPU当前可以向接口输出数据, 这个状态位可供CPU进行查询。
- 接口此时也可以向CPU发中断请求。CPU既可以用查询程序方式, 也可以用程序中断方式向接口输出数据。

3、并行通信接口的基本输入/输出工作过程

(2) 输出过程

- 当CPU将数据送到输出缓冲器后, 接口自动清除“输出准备好”状态位为0(表示输出缓冲器已满), 并将数据送往外设的数据线上, 同时, 接口将给外设发送“启动(选通)信号”来启动外设接收数据。
- ◆ 外设启动后,开始接收数据,并向接口发“数据输出应答”(“数据准备好”)信号(来自引脚,表示外设已取走数据, 接口可向外设传送数据)。
- ◆ 接口收到此信号,便将状态寄存器中的“输出准备好”状态位置1(表示接口输出缓冲器已空), 以便CPU输出下一个数据到输出缓冲器。

2 串行接口

发送缓冲器也做
输出移位寄存器

2.1 单片机的串行接口

- IAP15W4K58S4单片机具有4个采用UART工作方式的全双工串行通信接口(串口1,串口2,串口3,串口4)。
- 每个串口由2个数据缓冲器、1个(输入)移位寄存器、1个串行控制寄存器和一个波特率发生器等组成。
 - ◆ 每个串口的数据缓冲器由串行接收缓冲器和发送缓冲器构成，它们在物理上独立，既可以接收数据也可以发送数据，还可以同时发送和接收数据。
 - ◆ 接收缓冲器只能读出，不能写入，而发送缓冲器则只能写入，不能读出。它们共用一个地址号。

sfr SBUF =0x99;// 串口1数据寄存器	sfr S3BUF=0xAD;//串口3数据寄存器
sfr S2BUF=0x9B;//串口2数据寄存器	sfr S4BUF=0x85;//串口4数据寄存器

2.1 单片机的串行接口

- IAP15W4K58S4的串行口既可以用于串行异步通信，也可以构成同步移位寄存器。
- 如果在串行口的输入/输出引脚上加上电平转换器，可以方便地构成标准的RS-232接口。
- IAP15W4K58S4单片机的串行口有4种工作方式，有的工作方式的波特率是可变的。用户用软件编程的方法在串行控制寄存器中写入相应的控制字节，即可改变串行口的波特率和工作方式。
- IAP15W4K58S4单片机串口1对应的硬件部分是TxD和RxD, 串口2对应的硬件部分是TxD2和RxD2, 串口3对应的硬件部分是TxD3和RxD3, 串口4对应的硬件部分是TxD4和RxD4。

2.1 单片机的串行接口

- IAP15W4K58S4单片机串口1对应的引脚是TxD和RxD，串口2对应的引脚是TxD2和Rx2D。
- 通过设置寄存器AUXR1中的S1_S1和S1_S0，串口1可以在三个地方切换。
- 串口2可在2个地方切换, 由P_SW2的S2_S0位选择。
- 具体见 [3.2.2 单片机的引脚及功能](#)。

3.4.2 单片机的输入输出引脚 3. I/O口的复用功能

串口1可在3个地方切换,由S1_S1和S1_S0选择,如表3-12

寄存器	D7	D6	D5	D4	D3	D2	D1	D0
AUXR1	S1_S1	S1_S0	CCP_S1	CCP_S0	SPI_S1	SPI_S0	0	DPS
P_SW2	EAXSFR	DBLPWR	P31PU	P30PU		S4_S	S3_S	S2_S

S1_S1	S1_S0	串口1的切换引脚
0	0	串口1在[P3.0/RxD, P3.1/TxD]
0	1	串口1在[P3.6/RxD_2, P3.7/TxD_2],
1	0	串口1在[P1.6/RxD_3/XTAL2, P1.7/TxD_3/XTAL1], 串口1在P1口时要使用内部时钟
1	1	无效

- ◇ 串口2可以在2个地方切换, 由S2_S0控制位选择:
- ◆0: 串口2在[P1.0/RxD2, P1.1/TxD2]
 - ◆1: 串口2在[P4.6/RxD2_2, P4.7/TxD2_2]

3.4.2 单片机的输入输出引脚

3. I/O口的复用功能

寄存器	D7	D6	D5	D4	D3	D2	D1	D0
P_SW2	EAXSFR	DBLPWR	P31PU	P30PU		S4_S	S3_S	S2_S

串口3可以在两个地方切换，由S3_S控制位选择：

0：串口3在[P0.0/RXD3， P0.1/TXD3]；

1：串口3在[P5.0/RXD3_2， P5.1/TXD3_2]。

串口4可以在两个地方切换，由S4_S控制位选择：

0：串口4在[P0.2/RXD4， P0.3/TXD4]；

1：串口4在[P5.2/RXD4_2， P5.3/TXD4_2]。

2.1 单片机的串行接口

1、串口的寄存器

与串行接口1相关的寄存器有：

SCON、PCON、AUXR、SBUF、**TMOD**、**TL1**、**TH1**、**TCN**、**IE**、**IP**、CLK_DIV、P_SW2、SADEN和SADDR。

与串行接口2相关的寄存器有：

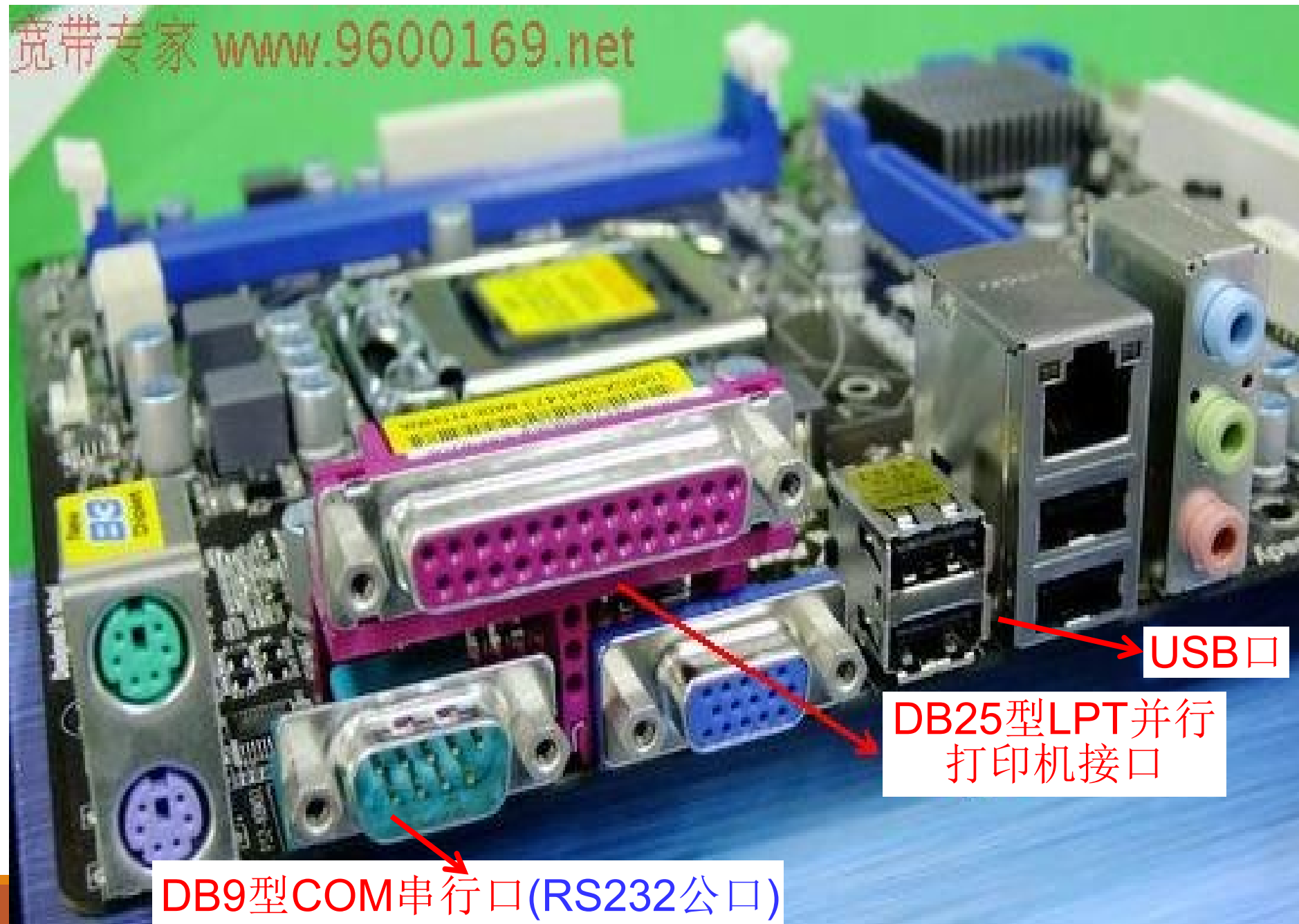
S2CON、S2BUF、T2H (TH2)、T2L(TL2)、AUXR、**IE2**、**IP2**和AUXR1。

与串口3相关的寄存器有：S3CON、S3BUF。

与串口4相关的寄存器有：S4CON、S4BUF。

2.2 RS232串行通信接口

- 在以计算机为中心的**数据采集与控制系统**中，通常需要用**单片机采集数据**，然后将数据用**异步串行通信方式**传给计算机；
- 要完成的控制命令由计算机通过串行通信方式传给单片机，由**单片机进行控制**。
- 计算机和单片机之间的串行通信一般采用**RS-232或RS-485**总线标准接口。这里介绍RS-232接口串行通信的设计方法。



DB9型COM串行口(RS232公口)

DB25型LPT并行
打印机接口

USB口

2.2 RS232串行通信接口

RS-232

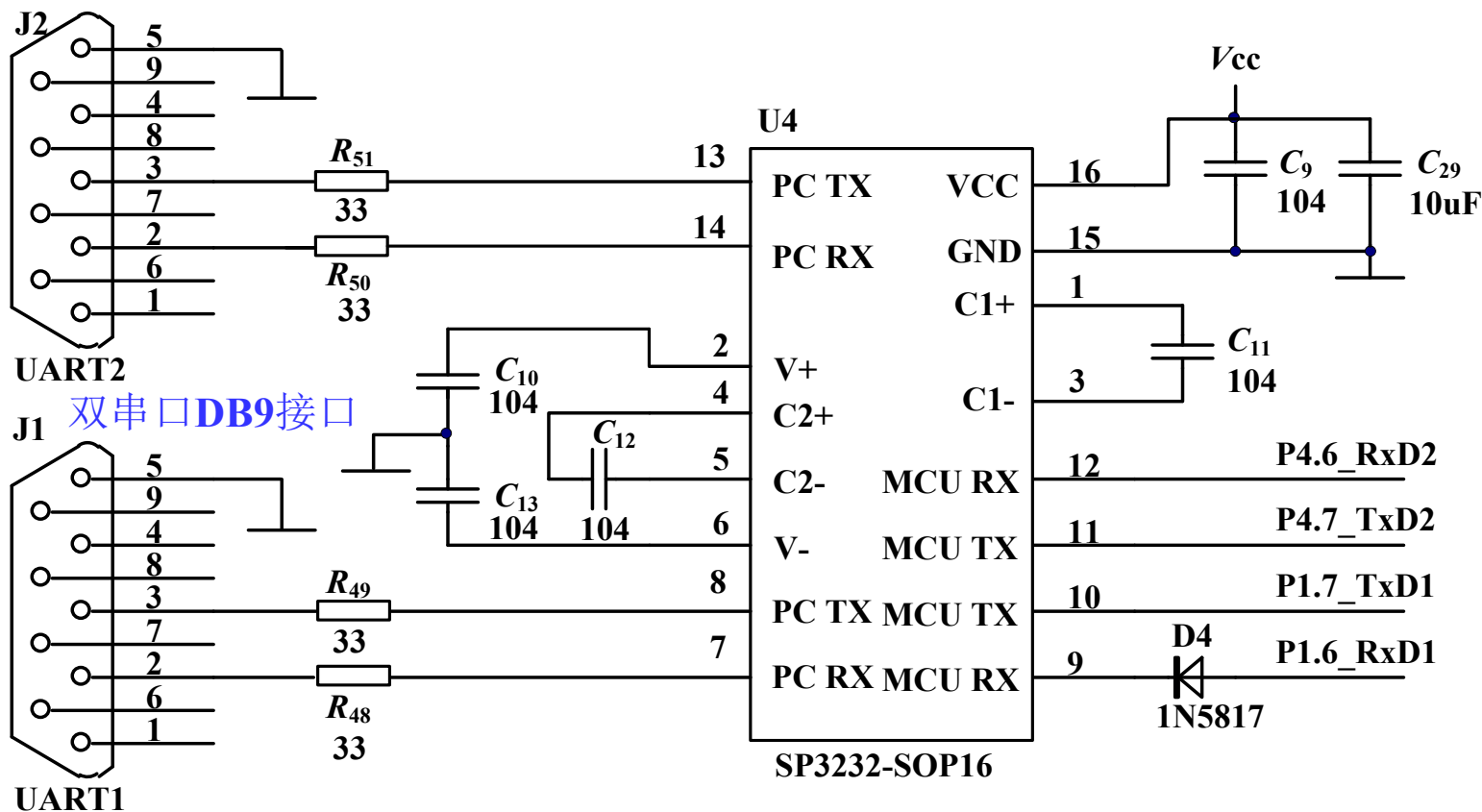
- 是早期为公用电话网络数据通信而制定的标准。
- 其逻辑电平与TTL/CMOS电平完全不同。逻辑“0”规定为+5~+15V之间，逻辑“1”规定为-5~-15V之间。
- 由于RS-232发送和接收之间有公共地，传输采用非平衡模式，因此共模噪声会耦合到信号系统中，标准中建议的最大通信距离为15米。

1、硬件接口设计

- ◇ 计算机串口是RS-232电平，而单片机串口是TTL电平。因此，须用电路实现TTL电平和RS-232电平转换。
- ◇ 常用电平转换芯片是MAX232(或兼容芯片SP3232等)，其用单电源(+5V)供电，RS232电平由内部电荷泵产生。

1、硬件接口设计

如图转换芯片SP3232将单片机的2个串口转换成2个RS-232 接口,使单片机可用计算机直接串行通信。



RS-232转换芯片SP-3232与单片机的硬件连接

1、硬件接口设计

目前，较新的个人计算机都没有了DB9串行口，特别是笔记本电脑，而USB接口较多。在这种情况下，我们可以使用USB转串口的芯片进行转换，

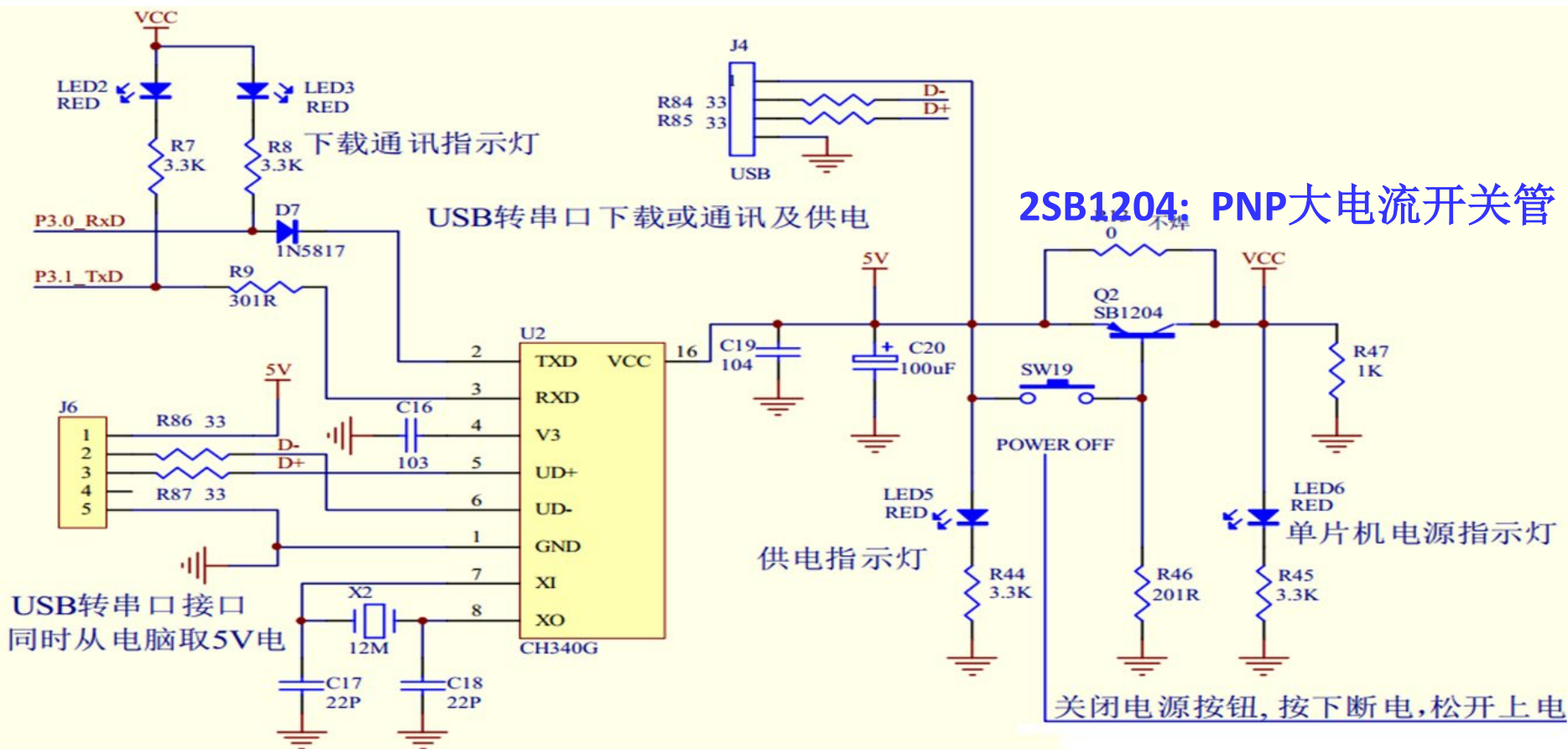
常见的USB转串口芯片有CH340T、CP210x等。CH340与单片机的接口电路见[图4-28](#)。

- ◇ USB转串口电路只是实现USB传输协议和UART传输协议之间的转换，对于用户而言，程序设计上没有本质的改变，相当于一个串口使用。
- ◇ 唯一需要注意的是，在计算机上编程选择串口的时候，串口号的选择要正确。

USB转串口下载或通讯及供电电路图

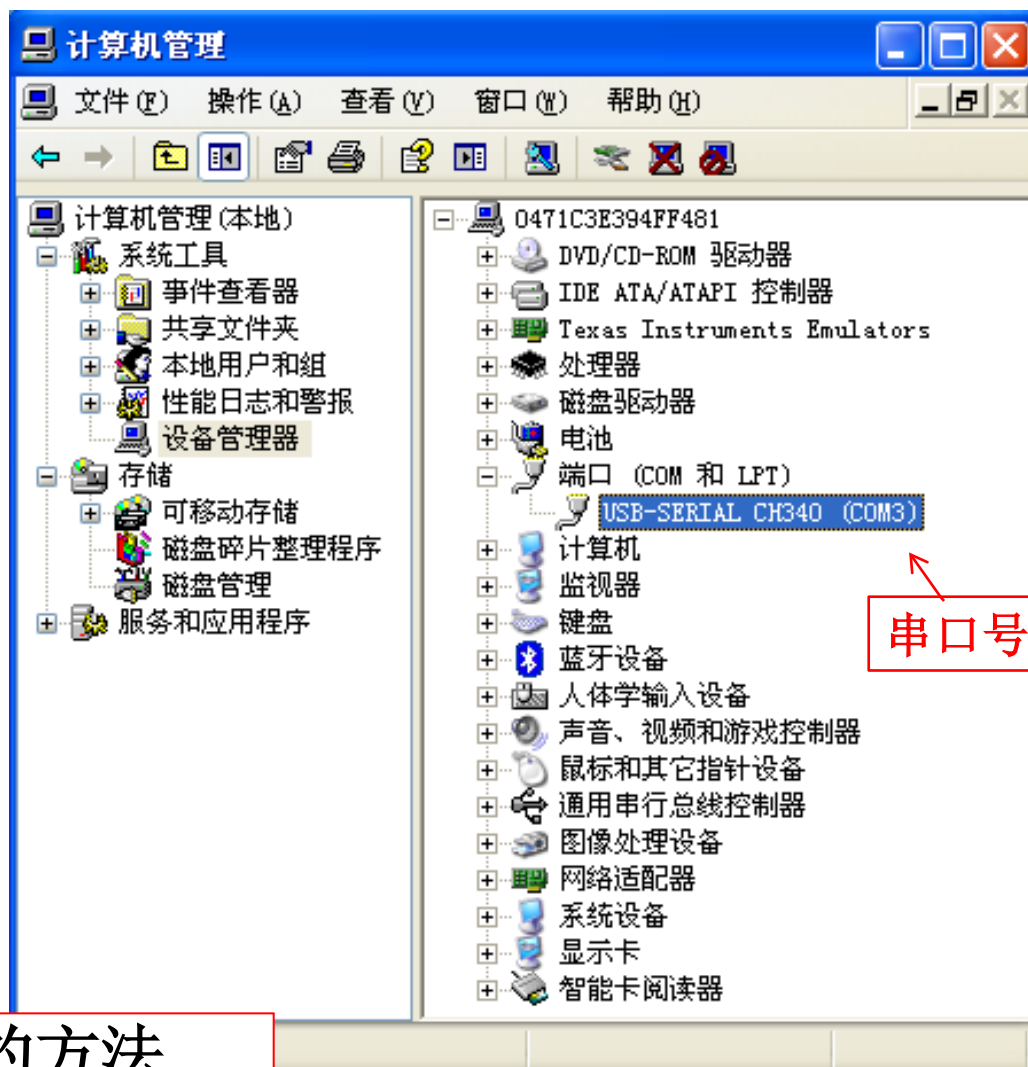
- (1) 硬件设置: 仿真方式为单CPU兼做仿真。电脑经USB转UART的接口与单片机连接。集成的转换电路图见图4-28。

CH340完成USB到UART的转换。用J4或J6连接电脑USB。



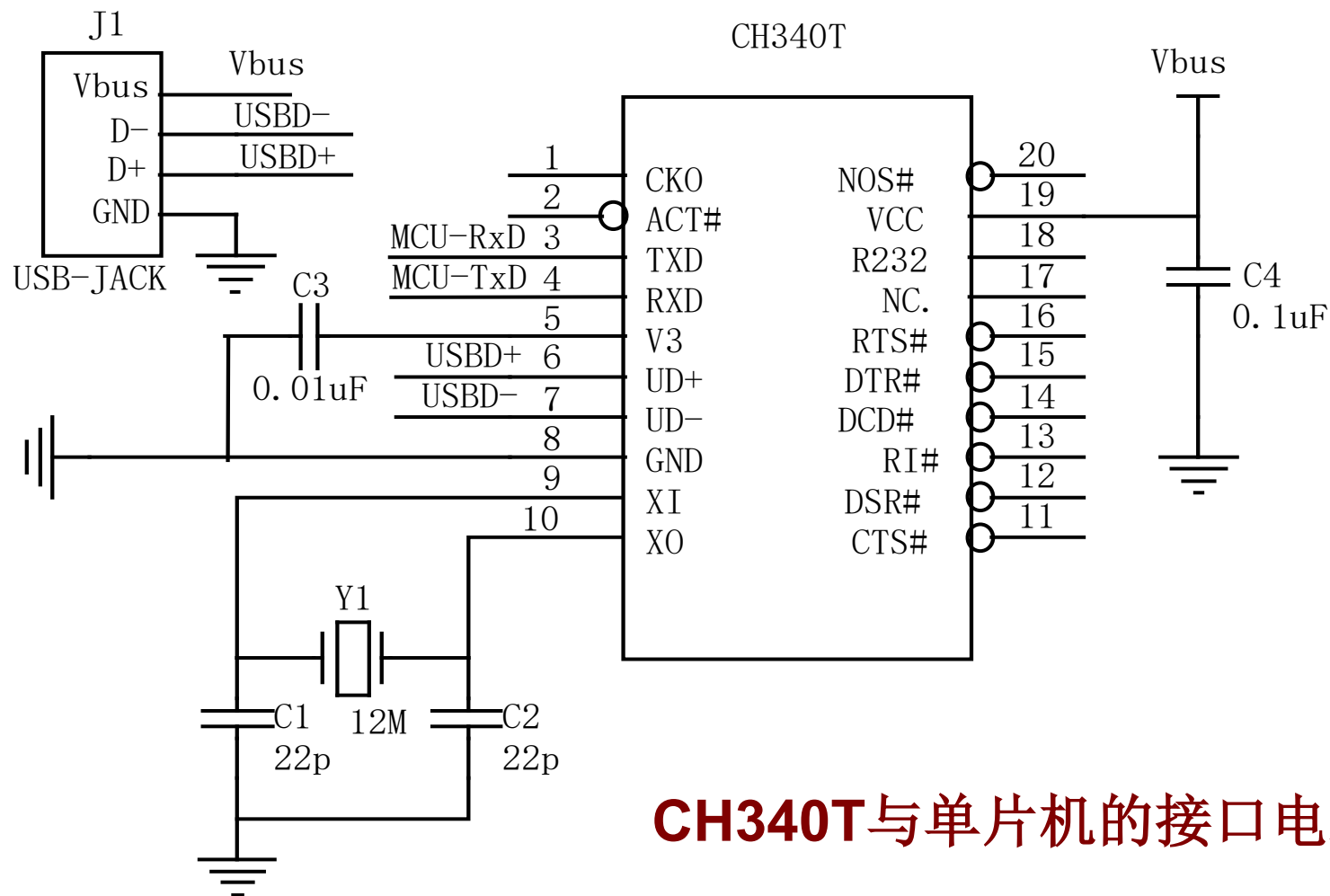
1、硬件接口设计

在计算机上编程选择串口的时候,串口号的选择要正确。



查找串口号的方法

CH340T与单片机的接口电路如图所示



CH340T与单片机的接口电路

CH340T管脚定义:

TXD:输出,串行数据输出(CH340R 型号为反相输出)

RXD:输入,串行数据输入,内置可控的上拉和下拉电阻

XI:输入, CH340T/R/G 晶体振荡的输入端,需要外接晶体及电容

XO:输出,CH340T/R/G 晶体振荡的输出端,需要外接晶体及电容

RST#:输入,CH340B外部复位输入,低电平有效,内置上拉电阻

UD+:USB信号,直接连到USB总线的D+数据线

UD-:USB信号,直接连到USB总线的D-数据线

CTS#:输入,MODEM联络输入信号,清除发送,低(高)有效

RTS#:输出,MODEM联络输出信号,请求发送,低(高)有效

DSR#:输入,MODEM联络输入信号,数据装置就绪,低(高)有效

DTR#:输出,MODEM联络输出信号,数据终端就绪,低(高)有效

RI#:输入,MODEM联络输入信号,振铃指示,低(高)有效

DCD#:输入,MODEM联络输入信号,载波检测,低(高)有效

CH340T管脚定义:

R232:输入,CH340T/R/G辅助RS232使能,高有效,内置下拉电阻

NOS#(16管脚芯片无):输入,禁止USB设备挂起,低电平有效,内置上拉电阻

ACT#(16管脚芯片无):输出,USB配置完成状态输出,低电平有效

IR#(16管脚芯片无):输入,CH340R串口模式设定输入,内置上拉电阻,低电平为SIR红外线串口,高电平为普通串口

CKO(16管脚芯片无):输出,CH340T时钟输出

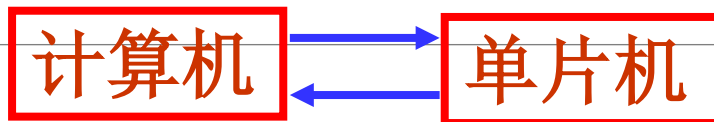
VCC:电源,正电源输入端,需要外接0.1uF电源退耦电容

GND:电源,公共接地端,直接连到USB总线的地线

V3:电源,在3.3V 电源电压时连接VCC输入外部电源, 在5V电源电压时外接容量为0.01uF退耦电容

2.2 RS232串行通信接口

RS-232转换接口



或USB转换接口

2、软件设计

软件设计往往因应用系统要求的不同而不同。软件设计分为上位机程序设计和单片机程序设计两部分。

如果仅仅为了测试串口的电路连接以及单片机通信程序设计正确与否，上位机程序可以直接使用现成的STC-ISP串口调试助手软件。

STC-ISP软件可从<http://www.stcmcu.com>中下载。

当然，也可以使用Visual C++等可视化程序开发环境自行设计。

2.2 RS232串行通信接口 [例]

下面通过一个实例，介绍计算机与单片机进行RS232通信的[硬件接口设计](#)和软件设计。

[例] 计算机向单片机发送一个数据, 单片机接收到数据后, 将接收到的数据按位取反后回发给计算机。

- 假设单片机的系统时钟为11.0592MHz，通信参数为“9600,n,8,1” (这是常见通信参数表示方法, 即波特率为9600bit/s, 无奇偶校验(n: none) (或E:Even, O:Odd), 8个数据位, 1个停止位)。
- 在计算机上显示从单片机发送过来的数据。在这种方式中，计算机通常称为上位机。

[例] 计算机向单片机送数据,单片机取反后回发计算机

下面**单片机程序设计**中，利用IAP15W4K58S4单片机的**串口2**和上位计算机通信，只能选择**定时器2**作为波特率发生器。

如果采用**串行口1**进行通信，则可以选择**定时器1**作为波特率发生器，也可以选择**定时器2**作为波特率发生器，程序设计方法与此类似，请读者自行实验。

(2) 串口2控制寄存器S2CON

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	S2SM0	0 -	S2SM2	S2REN	S2TB8	S2RB8	S2TI	S2RI

其中，S2SM0用于指定串口2的工作方式，如表所示：

S2SM0	工作方式	功能说明	波特率
0	方式0	8位UART, 波特率可变	(定时器2的溢出率)/4
1	方式1	9位UART, 波特率可变	(定时器2的溢出率)/4

◆当T2x12=1时, 定时器2的溢出率:

$$= \text{SYSclk} / (65536 - [\text{RL_TH2}, \text{RL_TL2}])$$

◆当T2x12=0时, 定时器2的溢出率:

$$= \text{SYSclk} / 12 / (65536 - [\text{RL_TH2}, \text{RL_TL2}])$$

◆RL_TH2, RL_TL2分别是T2H, T2L的重装载寄存器。

2.3 RS485串行通信接口

RS-485串行数据接口是为弥补RS-232通信距离短（15米）、速率低等缺点而产生的。

在RS-422基础上制定的标准，增加了多点、双向通信能力，即允许多个发送器连接到同一条总线上，同时增加了发送器的驱动能力和冲突保护特性。

RS-485标准只规定了平衡发送器和接收器的电特性，而没有规定接插件、传输电缆和应用层通信协议。

2.3 RS485串行通信接口

- RS-485数据信号采用差分传输方式，也称作平衡传输，它使用一对双绞线，将其中一线定义为A，另一线定义为B。
- A、B之间(A-B)的正电平在 $+2V \sim +6V$ ，表示逻辑状态“1”；负电平在 $-2V \sim -6V$ ，表示逻辑状态“0”。
- RS-485标准的最大传输距离约为1200米，最大传输速率为10Mbps。

2.3 RS485串行通信接口

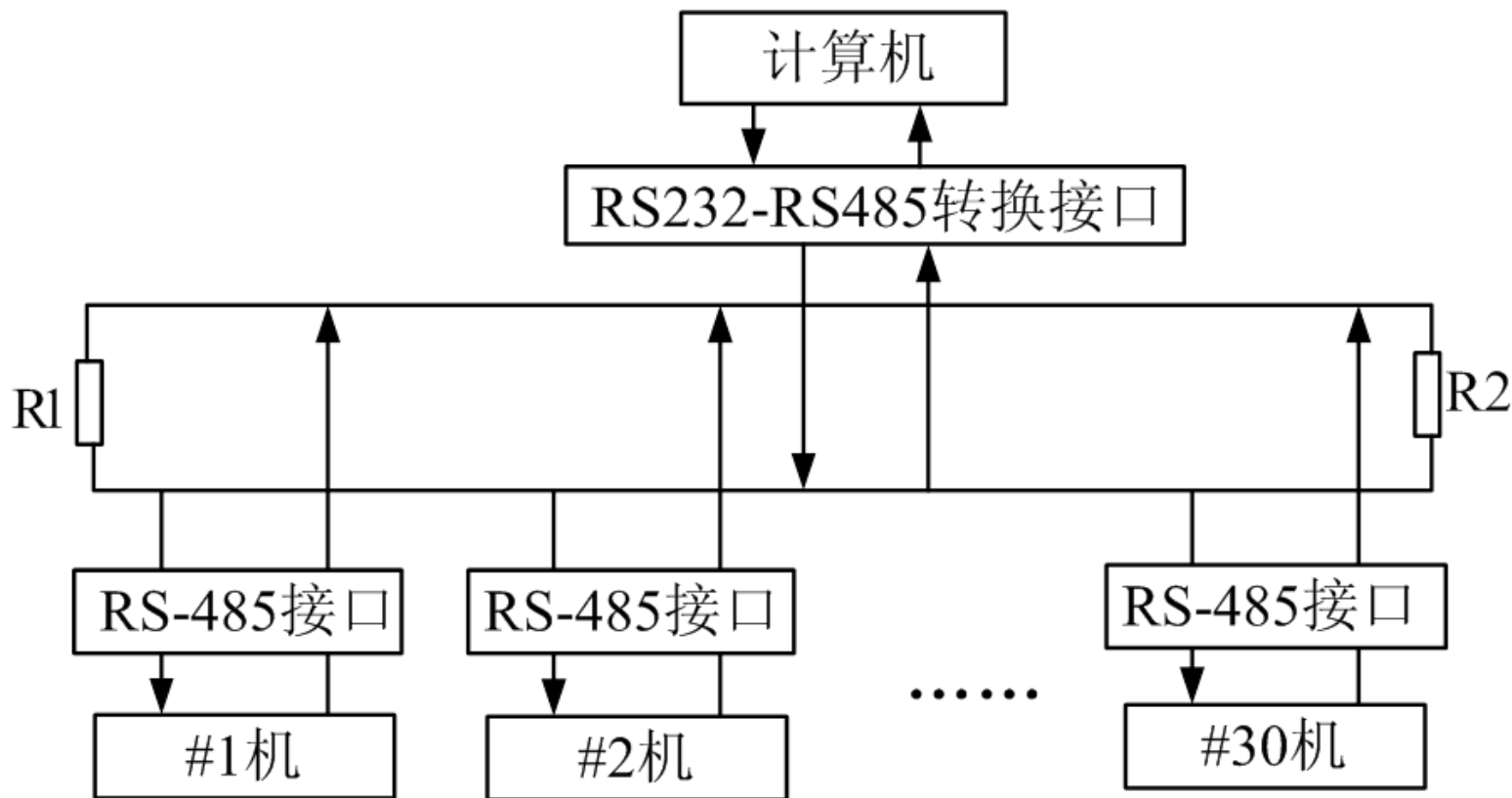
- RS-485网络采用平衡双绞线作为传输介质。
平衡双绞线的长度与传输速率成反比，只有在20 kbps速率以下，才可能使用规定最长的电缆长度。只有在很短的距离下才能获得最高速率传输。
- 一般来说，100米长的双绞线最大传输速率仅为1Mbps。如果采用光电隔离方式，则通信速率一般还会受到光电隔离器件响应速度的限制。

2.3 RS485串行通信接口

- 利用RS-485标准，可以建立一个相对经济、具有高噪声抑制、高传输速率的通信平台，该平台同时具有传输距离远、宽共模范围、控制方便等优点。
- 目前，在工程应用的现场网络中，RS-485半双工异步通信总线被广泛应用在集中控制枢纽与分散控制单元之间通信的场合。

2.3 RS485串行通信接口

一台计算机作主机, 通过RS-485连接现场的控制单元, 系统结构如图所示。



主从结构的RS-485网络结构图

2.3 RS485串行通信接口

RS-485接口芯片可用半双工传输的MAX3082 (或其他RS485接口芯片, 如MAX485, MAX487)。

MAX3082的结构及典型半双工通信电路图如图所示。

- ◇ 单片机接收数据时, 应通过指令将P1.0清0; 单片机发送数据时, 应通过指令将P1.0置1。

A : 485差分信号的正向端;

B : 485差分信号的反向端;

VCC :电源端;

GND :接地端。

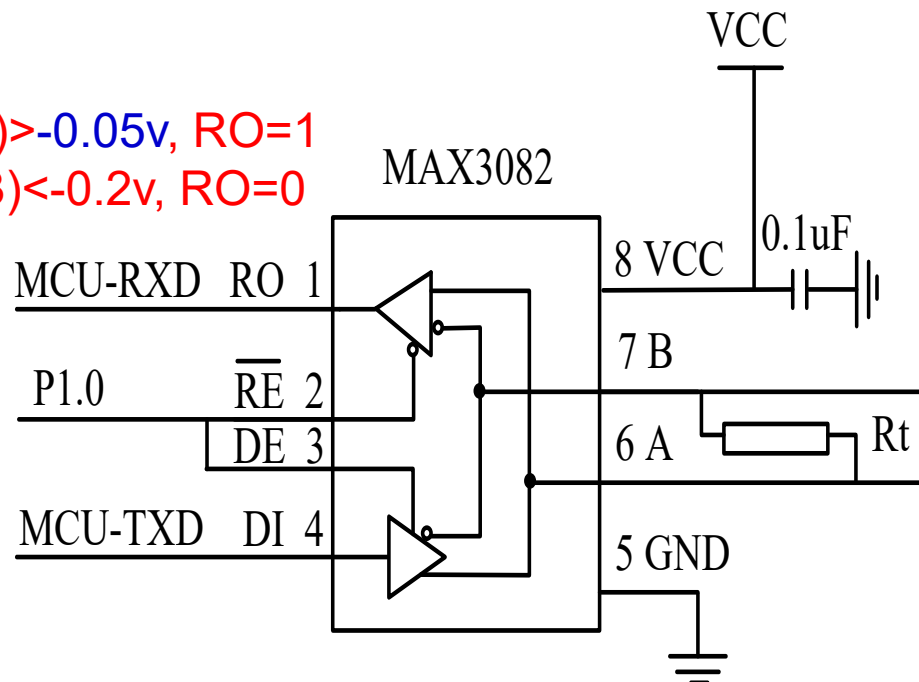
RE :接收允许端低电平有效;

DE :发送允许端高电平有效;

RO:接收数据的TTL电平输出端;

DI :发送数据的TTL电平输入端;

$(A-B) > -0.05V$, RO=1
 $(A-B) < -0.2V$, RO=0



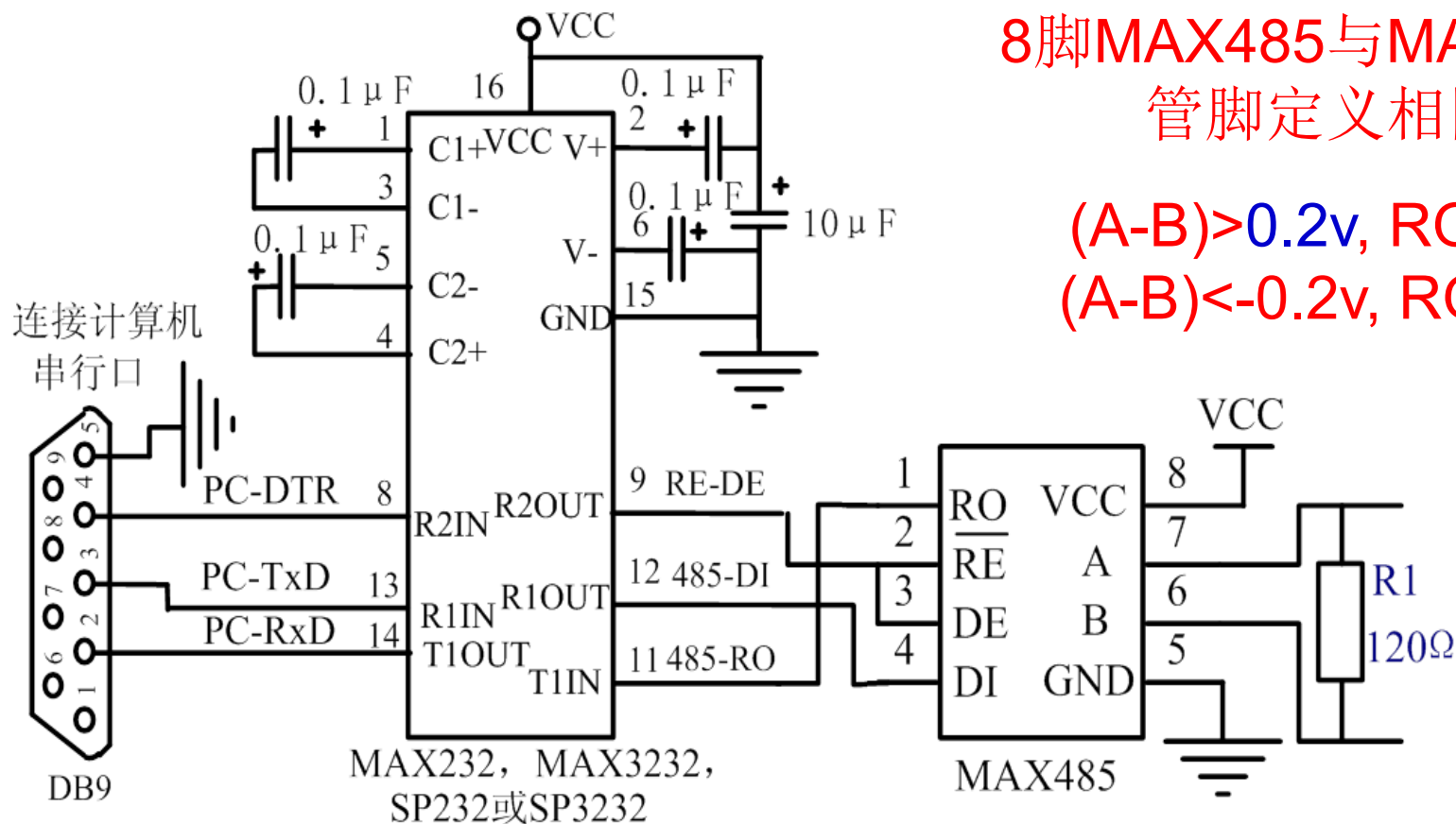
MAX3082的结构及典型的半双工通信电路图

2.3 RS485串行通信接口

连接计算机的RS-232和RS-485转换电路如图所示。

8脚MAX485与MAX3082
管脚定义相同

$(A-B) > 0.2V$, $RO=1$
 $(A-B) < -0.2V$, $RO=0$



连接计算机的**RS-232**和**RS-485**转换电路

MAX485

2.4 SPI通信接口

1、SPI接口简介

IAP15W4K58S4集成了串行外设接口(Serial Peripheral Interface, 简称SPI)。

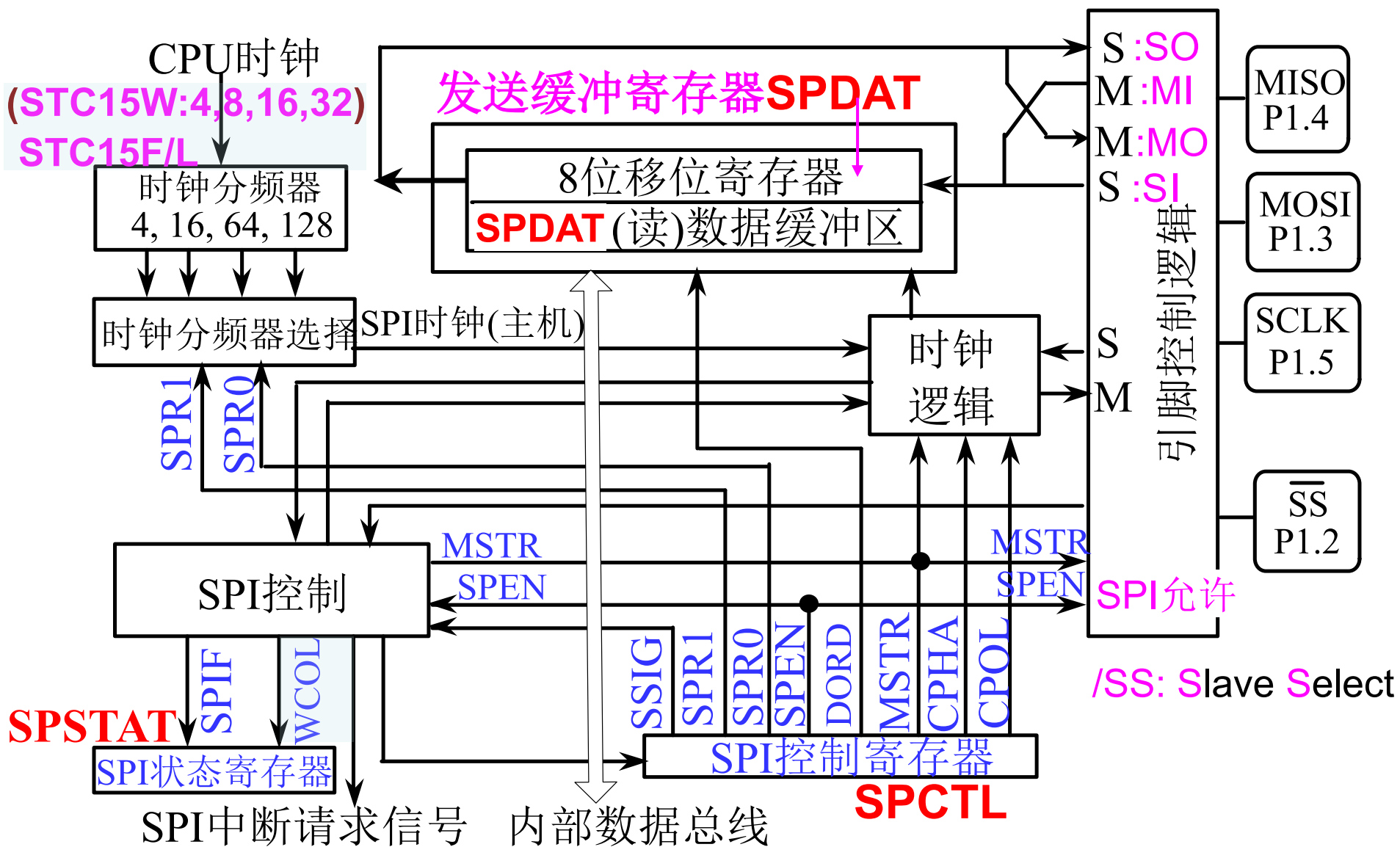
SPI接口既可以和其他微处理器通信，也可以与具有SPI兼容接口的器件，如存储器、A/D转换器、D/A转换器、LED或LCD驱动器等进行同步通信。

◇SPI接口有两种操作模式：主模式和从模式

12



2、IAP15W4K58S4单片机的SPI接口的结构



IAP15W4K58S4单片机的SPI接口结构说明

- SPI的**核心是一个8位移位寄存器和数据缓冲器**, 数据可以同时发送和接收。在SPI数据的传输过程中, 发送和接收的数据都存储在缓冲器中。
- **对于主模式**, 若要发送一个字节数据, 只需将这个数据写到**SPDAT**寄存器中。主模式下/**SS**信号不是必须的。
- **在从模式下**, 必须在/**SS**信号变为有效并接收到合适的时钟信号后, 方可进行数据的传输。在从模式下, 如果一个字节传输完成后, /**SS**信号变为高电平, 这个字节立即被硬件逻辑标志为接收完成, SPI接口准备接收下一个数据。
- 任何SPI控制寄存器的改变将**复位SPI接口**, 并清除相关寄存器。

3、SPI接口的数据通信

(1) SPI接口的信号

MISO/P1.4, MOSI/P1.3, SCLK/P1.5, /SS /P1.2 共4根信号线构成SPI接口。
SPI接口的引脚可以切换。

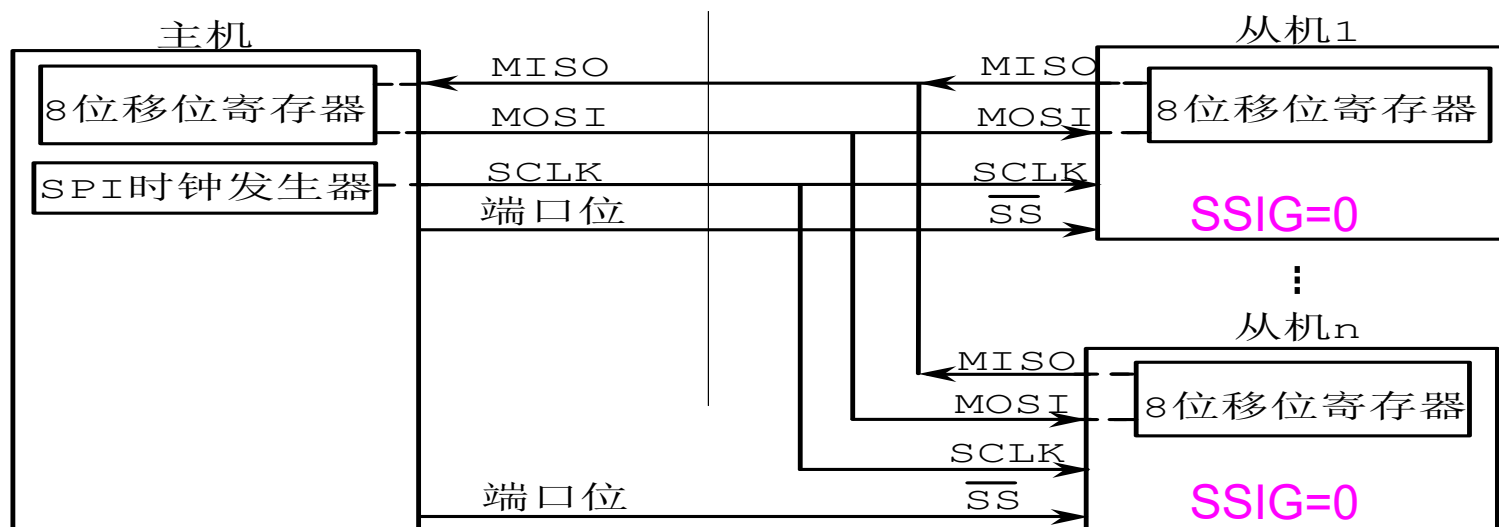
◇MOSI (Master Out Slave In, 主出从入)

- ◆主器件的输出和从器件的输入，用于主器件到从器件的串行数据传输。
- ◆根据SPI规范，多个从机共享一根MOSI信号线。
- ◆在时钟边界的前半周期，主机将数据放在MOSI信号线上，从机在该边界处获取该数据。

(1) SPI接口的信号

MISO (Master In Slave Out, 主入从出)

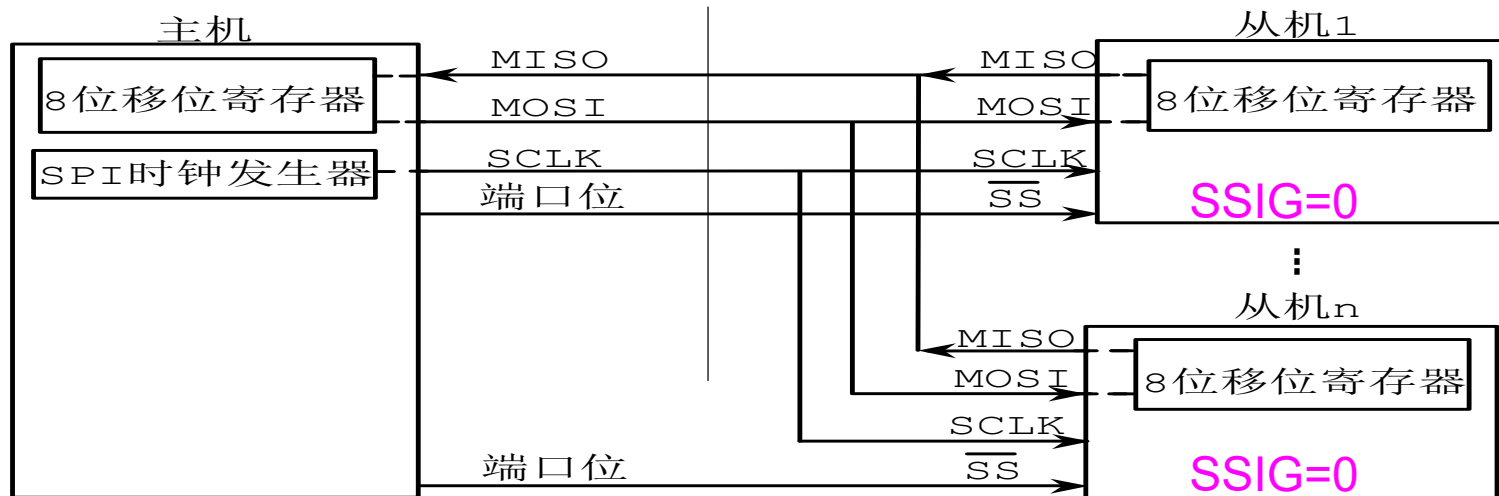
- 从器件的输出和主器件的输入。用于实现从器件到主器件的数据传输。
- SPI规范中，一个主机可连接多个从机，因此，**主机的MISO信号线会连接到多个从机上**，或者说，多个从机共享一根MISO信号线。
- 当主机与一个从机通信时，其他从机应将其**MISO引脚驱动置为高阻状态**。



(1) SPI接口的信号

SCLK (SPI Clock, 串行时钟信号)

- 串行时钟信号是主器件的输出和从器件的输入，用于同步主器件和从器件之间在MOSI和MISO线上的串行数据传输。
- 当主器件启动一次数据传输时，自动产生8个SCLK时钟周期信号给从机。在SCLK的每个跳变处（上升沿或下降沿）移出一位数据。
- 一次数据传输可传输一个字节的的数据。



(1) SPI接口的信号

SCLK、MOSI和MISO通常用于将两个或更多个SPI器件连接在一起。

- 数据通过MOSI由主机传送到从机，通过MISO由从机传送到主机。
- SCLK信号在主模式时为输出，在从模式时为输入。

◆ 如果SPI接口被禁止，即特殊功能寄存器SPCTL中的SPEN=0（复位值），这些管脚都可作为I/O口使用。

(1) SPI接口的信号 $\overline{SS}$

(Slave Select, 从机选择信号)

- 这是一个输入信号。主器件用它来选择处于从模式的SPI模块。
- 在主模式下, SPI接口就是主机(只允许一个主机), 不存在主机选择问题, 该模式下 $\overline{SS}$ 不是必须的。主模式下通常将主机的 $\overline{SS}$ 引脚通过10k Ω 电阻上拉至高电平。每个从机的 $\overline{SS}$ 接主机的I/O口, 由主机控制电平高低, 以便主机选择从机。
- 在从模式下, 不论发送还是接收, $\overline{SS}$ 信号必须有效。因此在一次数据传输开始之前必须将 $\overline{SS}$ 拉为低电平。SPI主机可用I/O口选择一个SPI器件(使 $\overline{SS}=0$)作为当前从机。

可置SSIG位为1, $\overline{SS}$ 脚被忽略

SPI控制寄存器SPCTL. 7(SSIG)

SPCTL 寄存器	D7	D6	D5	D4	D3	D2	D1	D0
	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

①SSIG: 引脚忽略控制位。

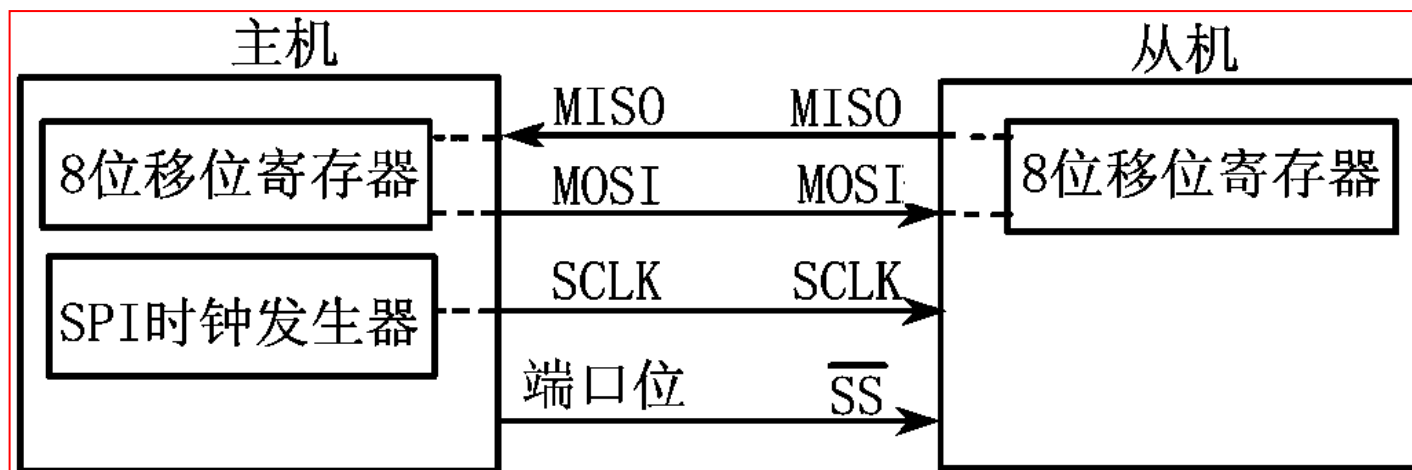
1: 由MSTR位确定器件为主机(MSTR=1)还是从机。

0: 由 $\overline{SS}$ 引脚的低电平, 决定从机被选中, 即使主机也变为从机。可通过控制 $\overline{SS}$ 脚为低电平, 使其被选为从机。

(1) SPI接口的信号 $\overline{SS}$

(Slave Select, 从机选择信号)

- 这是一个输入信号。主器件用它来选择处于从模式的SPI模块。
- 在主模式下, SPI接口就是主机(只允许一个主机), 不存在主机选择问题, 该模式下/ $\overline{SS}$ 不是必须的。主模式下通常将主机的/ $\overline{SS}$ 引脚通过10k Ω 电阻上拉至高电平。每个从机的/ $\overline{SS}$ 接主机的I/O口, 由主机控制电平高低, 以便主机选择从机。
- 在从模式下, 不论发送还是接收, / $\overline{SS}$ 信号必须有效。因此在一次数据传输开始之前必须将/ $\overline{SS}$ 拉为低电平。SPI主机可用I/O口选择一个SPI器件(使/ $\overline{SS}$ =0)作为当前从机。



(1) SPI接口的信号

SPI从器件通过其/SS脚是否为低确定是否被选择。若满足下面条件之一,可被忽略:

- 若SPI功能被禁止,即SPEN位为0(复位值);
- 若SPI配置为主机,即MSTR位为1,并且P1.2/SS配置为输出(P1M0.2=1, P1M1.2=0);
- 若/SS脚被忽略,即SSIG位为1,该脚用于I/O口功能。

注:即使SPI被配置为主机(MSTR=1),仍然可通过拉低/SS脚配置为从机(如果P1.2配置为输入且SSIG=0),此时MSTR=1→0,置位SPIF (SPSTAT.7) (自动置位,表示“模式改变”)。

SPCTL 寄存器	D7	D6	D5	D4	D3	D2	D1	D0
	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

3、SPI接口的数据通信

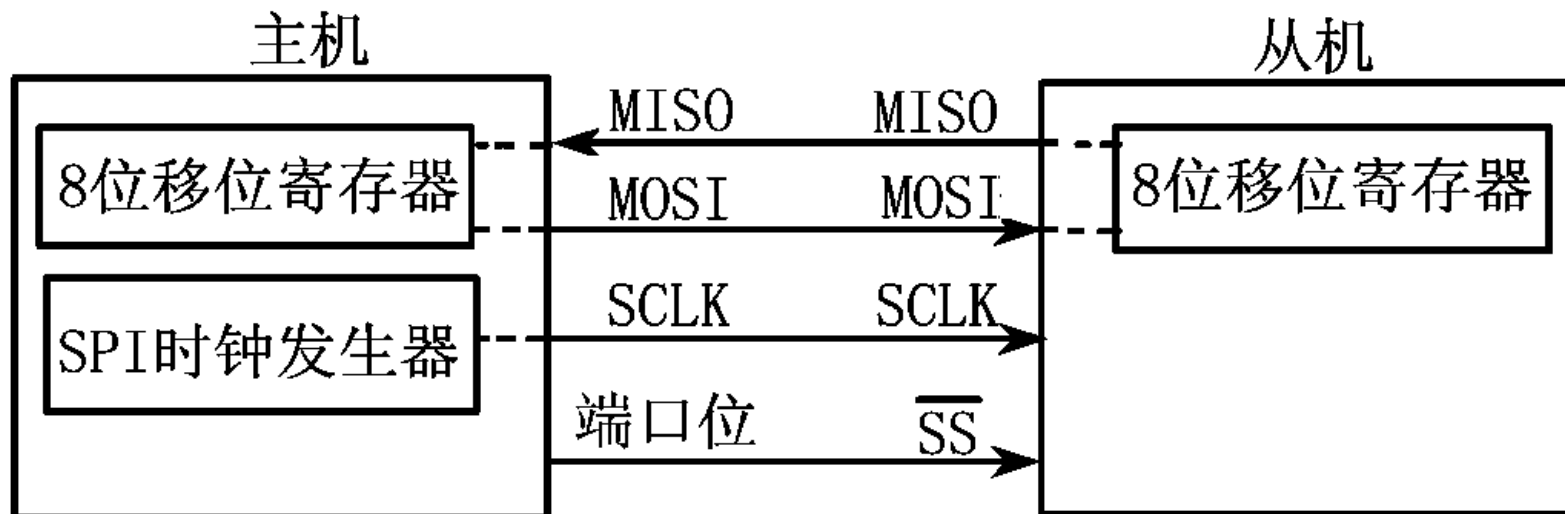
(2) SPI接口的数据通信方式

IAP15W4K58S4单片机SPI接口的数据通信方式有3种：

- 单主机-单从机方式
- 双器件方式（器件可互为主机和从机）
- 单主机-多从机方式。

1) 单主机-单从机方式

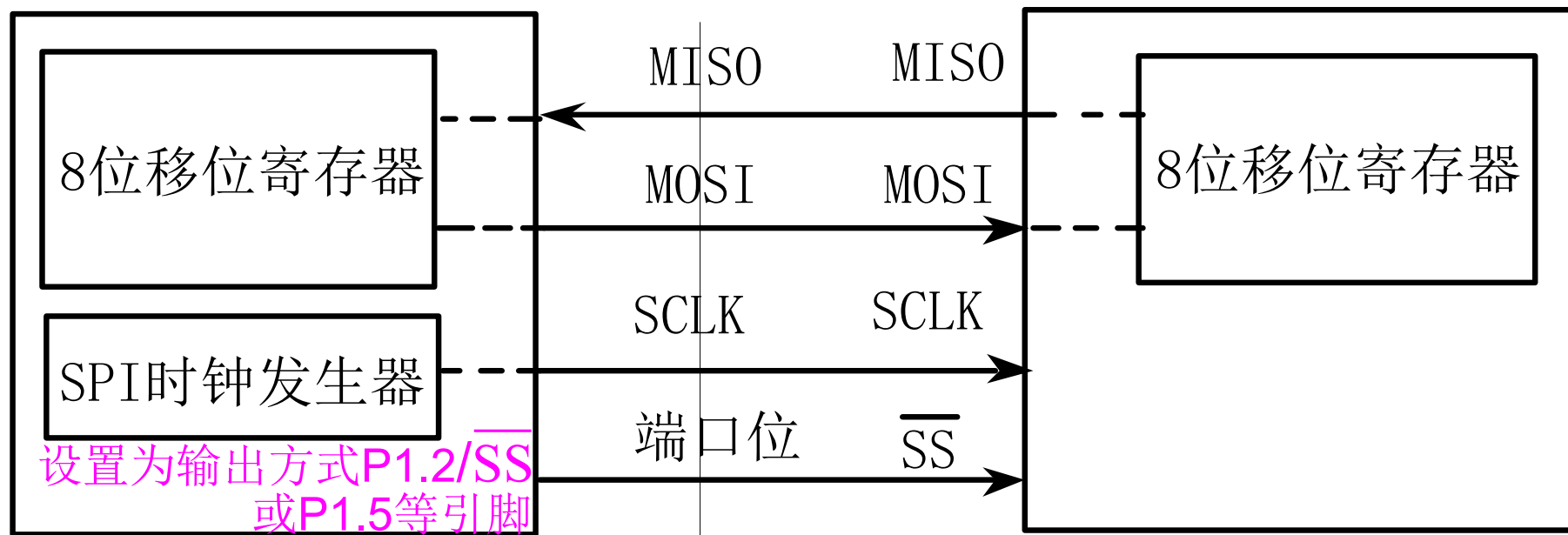
◇连接如图所示。



1) 单主机-单从机方式

图中, 从机SSIG(SPCTL.7)为0,不忽略 $\overline{SS}$ 用来选择从机。
SPI主机可用任何端口位(含P1.2/ $\overline{SS}$)来控制从机 $\overline{SS}$ 脚。

主机 **SPI接口的单主机-单从机连接方式** 从机



SPCTL寄存器	D7	D6	D5	D4	D3	D2	D1	D0
	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

1) 单主机-单从机方式

主机SPI与从机SPI的8位移位寄存器连成一个循环16位移位寄存器。

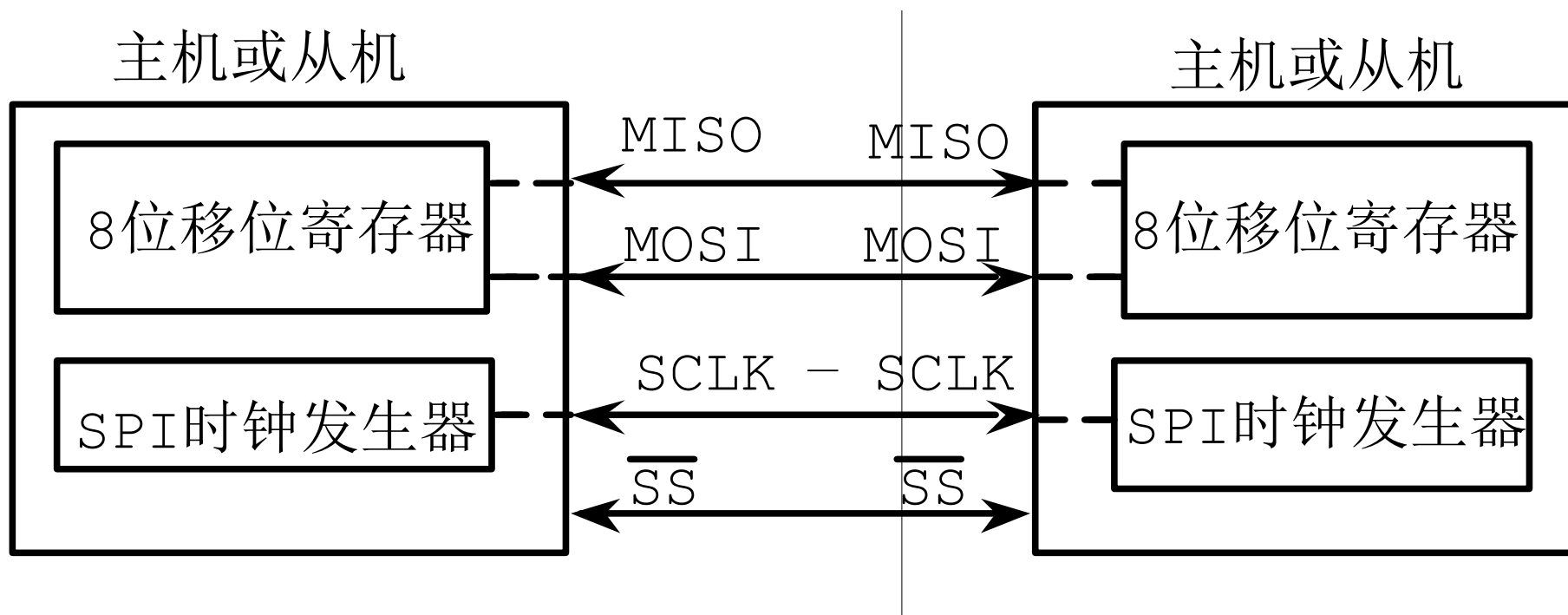
当主机向SPDAT写一字节时,就启动一连串8位移位通信过程: 主机SCLK脚向从机SCLK脚发一串脉冲, 在该串脉冲驱动下, 主机SPI的8位移位寄存器中数据移到从机8位移位寄存器中。同时从机8位移位寄存器中数据移到主机SPI的8位移位寄存器中。由此, 主机既可向从机发送数据, 又可由从机中读取数据。



(2) SPI接口的数据通信方式

2) 双器件方式

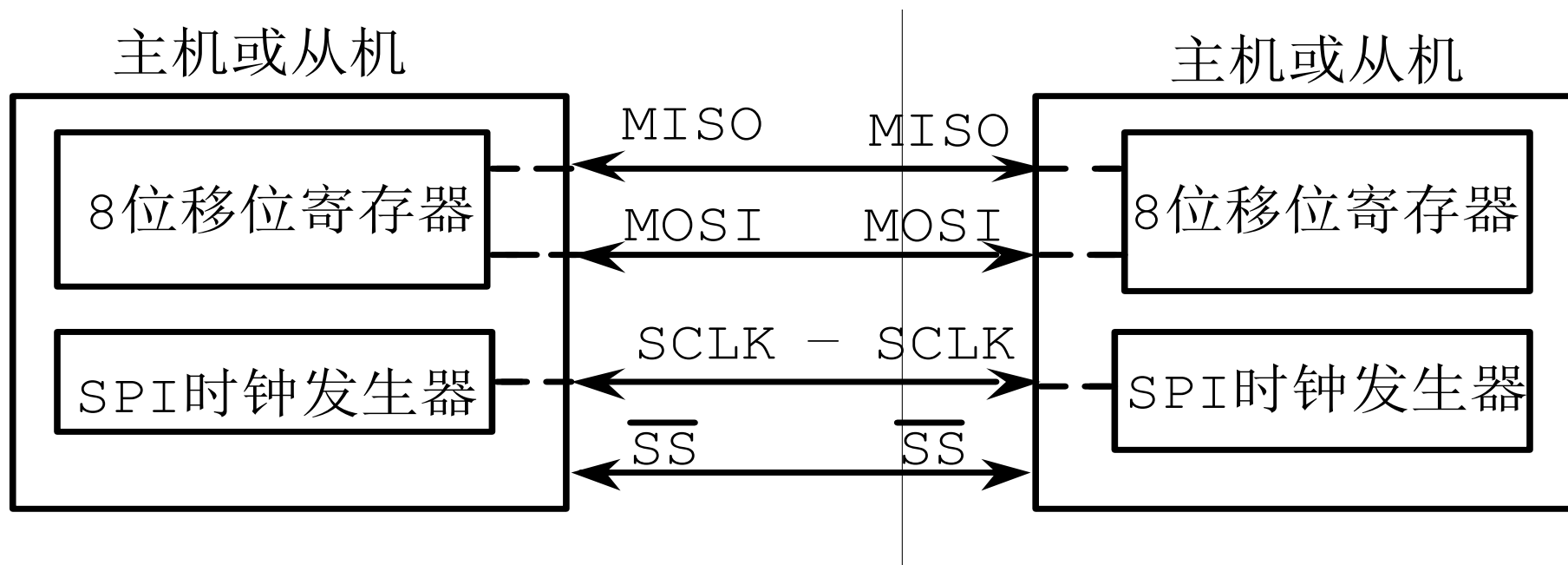
双器件方式也称互为主从方式，其连接方式如图所示。



SPI接口的双器件连接方式

2) 双器件方式

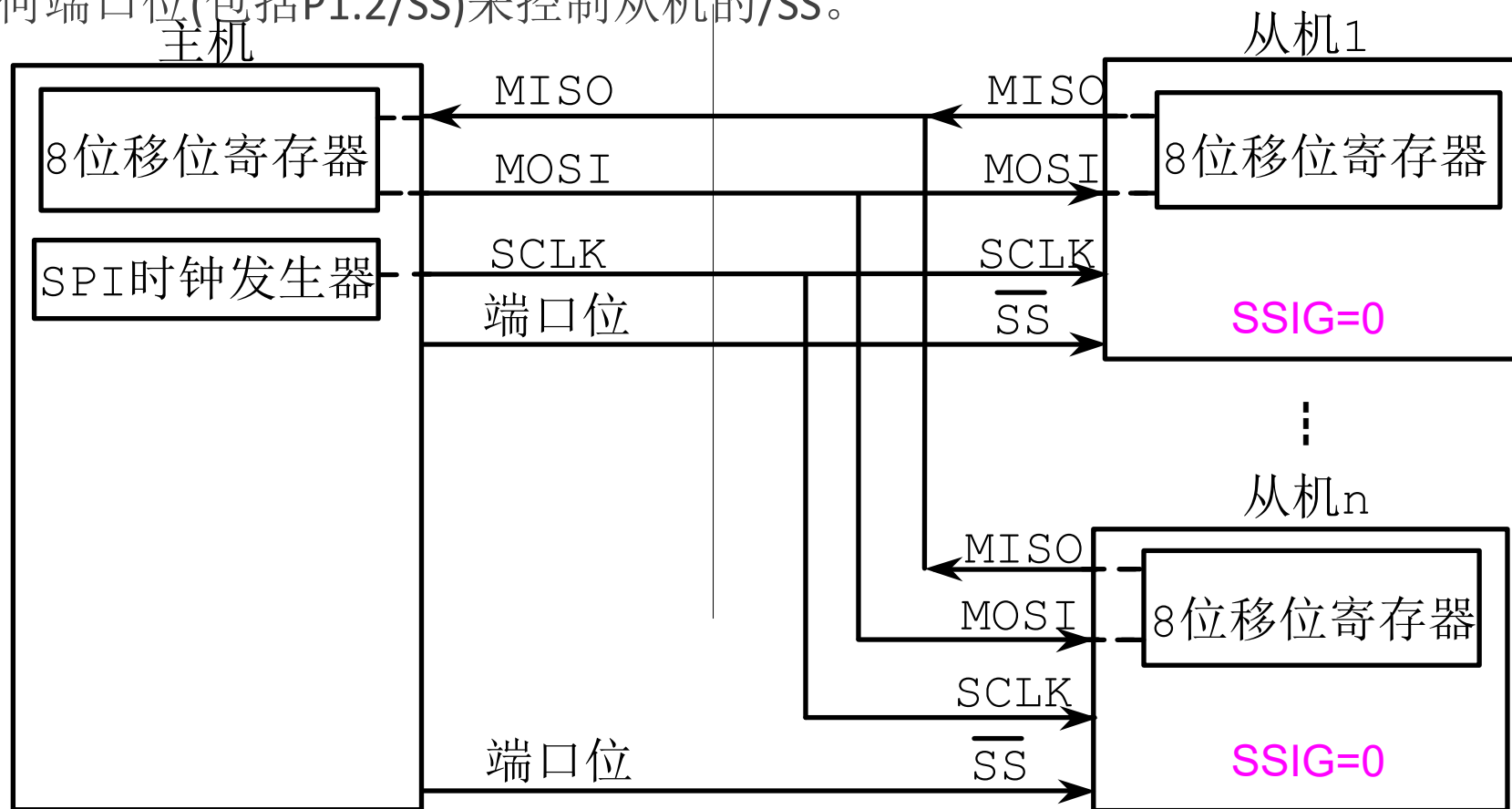
图中, 两个器件可互为主从。当没有发生SPI操作时, 两个器件都可配置为主机(MSTR=1), 将SSIG清零并将P1.2(SS)配置为准双向模式或输入模式。当其中一个器件启动传输时, 可将P1.2配置为输出并驱动为低电平, 这样就强制另一个器件变为从机。



(2) SPI接口的数据通信方式

3) 单主机-多从机方式

图中, 从机的**SSIG (SPCTL.7)**为0, 从机通过对应的/SS信号被选中。SPI主机可使用任何端口位(包括P1.2/SS)来控制从机的/SS。



SPI接口的单主机-多从机连接方式

(2) SPI接口的数据通信方式

主机模式的配置 主机和从机的选择

STC15单片机进行SPI通信时,主机和从机的选择由**SPEN**, **SSIG**, 引脚**P1.2(SS)**和**MSTR**联合控制; 主机和从机的选择如表所示。

SPEN	SSIG	/SS P1.2	MSTR	主或从 模式	MISO P1.4	MOSI P1.3	SCLK P1.5	备 注
0	X	P1.2	X	SPI功能 禁止	P1.4	P1.3	P1.5	SPI禁止:P1.2/P1.3/P1.4 /P1.5 作为普通I/O口使用
1	0	0	0	从机模式	输出	输入	输入	选择作为从机
1	0	1	0	从机模式 未被选中	高阻	输入	输入	未被选中。MISO为高阻状态, 以避免总线冲突
1	0	0	1→0	从机模式	输出	输入	输入	/SS配置为准双向口或输入模式, /SS若被驱动为低电平且SSIG=0, MSTR位自动清0。
1	0	1	1	主(空闲)	输入	高阻	高阻	主机空闲时MOSI和SCLK为 高阻态以避免总线冲突。用户须 将SCLK上拉或下拉(根据CPOL取值) 以避免SCLK出现悬浮状态。
				主(激活)		输出	输出	作为主机激活时, MOSI 和 SCLK 为推挽输出
1	1	P1.2	0	从	输出	输入	输入	
			1	主	输入	输出	输出	

(3) SPI接口的数据通信过程

- 在SPI通信中，数据传输总是由主机启动的。如果SPI使能($SPEN=1$)，主机对SPI数据寄存器的写操作将启动SPI时钟发生器和数据的传输。在数据写入SPDAT之后的半个到一个SPI位时间后，数据将出现在MOSI引脚。
- 需要注意的是，主机可以通过将对应器件的/SS引脚驱动为低电平实现与之通信。写入主机SPDAT寄存器的数据从MOSI脚移出并发送到从机的MOSI引脚。同时，从机SPDAT寄存器的数据从MISO引脚移出并发送到主机的MISO引脚。



- 主机和从机CPU的两个移位寄存器可以看作是一个16位循环移位寄存器。当数据从主机移位传送到从机的同时，数据也以相反的方向移入。这意味着在一个移位周期中，主机和从机的数据相互交换。

(3) SPI接口的数据通信过程

- 传输完一个字节后，SPI 时钟发生器停止，传输完成标志(SPIF)置位并产生一个中断(若SPI中断使能)。

(4) SPI中断

- 如果允许SPI中断，发生SPI中断时，CPU就会跳转到中断服务程序的入口地址004BH处执行中断服务程序。
- 注意：在中断服务程序中，必须把SPI中断请求标志清0。
SPIF标志通过软件向其写入"1" 而清0。

SPI状态寄存器 (SPSTAT)

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SPIF	WCOL	-	-	-	-	-	-

(3) SPI接口的数据通信过程

- 作从机时, 若CPHA=0, SSIG必须为0, /SS引脚须取反且在每个连续的串行字节之间重新设置为高电平。
- 若SPDAT寄存器在/SS有效(低电平)时执行写操作, 将导致一写冲突错误, WCOL (SPSTAT.6)标志被置1。CPHA=0且SSIG=01时的操作未定义。
- 当CPHA=1时, SSIG可以为1或0。如果SSIG=0, /SS引脚可在连续传输之间保持有效(即一直为低电平)。当系统中只有一个SPI主机和一个SPI从机时, 这是首选配置。

SPCTL								
位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

3、SPI接口的数据通信

通过/SS改变模式

如果SPEN=1，SSIG=0且MSTR=1，SPI使能为主机模式。

/SS引脚可配置为输入或准双向模式。这种情况下，另外一个主机可将该引脚驱动为低电平，从而将该器件选择为SPI从机并向其发送数据。

SPI控制寄存器SPCTL

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

(4) 通过/SS改变模式

为避免争夺总线，SPI系统执行以下动作：

- MSTR清0并且CPU变成从机。这样SPI就变成从机。MOSI和SCLK强制变为输入模式，而MISO则变为输出模式。
- SPSTAT的SPIF标志位置位。如果SPI中断已被使能，则产生SPI中断。

用户程序必须一直对MSTR位进行检测，如果该位被一个从机选择清零而用户想继续将SPI作为主机，就必须重新置位MSTR，否则将进入从机模式。

3、SPI接口的数据通信

(5) 写冲突 **SPDAT(接收发送共用同址)及8位移位寄存器**

SPI在发送时为单缓冲, 接收时为双缓冲。这样在前一次发送尚未完成之前, 不能将新数据写入移位寄存器。

当在发送过程中对数据寄存器进行写操作时, **WCOL**位会置1以指示数据冲突。在这种情况下, 当前发送的数据继续发送, 而新写入的数据将丢失。

◇ **WCOL**可通过软件向其写入“1”清0。

SPSTAT.6(WCOL)

◇ 接收数据时, 8位接收完毕后传送到并行读数据缓冲区, 这样将释放移位寄存器以进行下一数据的接收。但必须在下个字符完全移入之前从数据寄存器中读出接收到的数据, 否则, 前一个接收数据将丢失。

SPDAT

(5) 写冲突

当对主机或从机进行写冲突检测时，主机发生写冲突的情况很罕见，因主机拥有数据传输的完全控制权。但从机有可能发生写冲突，因为当主机启动传输时，从机无法进行控制。

接收数据时，接收到的数据传送到一个并行读数据缓冲区，这样将释放移位寄存器以进行下一数据的接收。但必须在下个字符完全移入之前从数据寄存器中读出接收到的数据，否则，前一个接收数据将丢失。

WCOL可通过软件向其写入“1”清0。

(6) 数据格式

数据格式与SPI控制寄存器SPCTL的CPHA和CPOL有关:

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

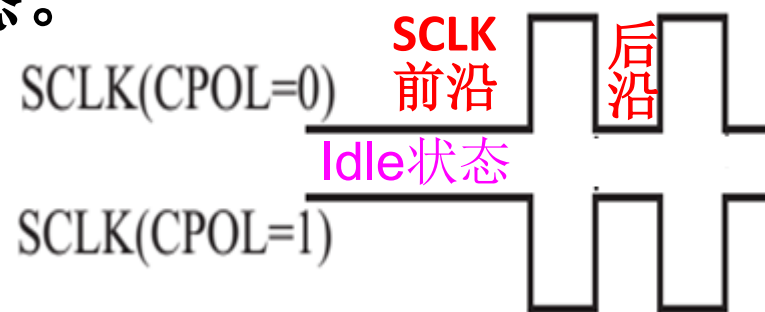
时钟相位控制位CPHA用于设置采样和改变数据的时钟SCLK的边沿。

时钟极性控制位CPOL用于设置时钟SCLK的极性。

➤SPI接口时钟信号线SCLK有Idle和Active两种状态:

Idle: 不进行数据传输时(或数据传完后)SCLK所处状态;

Active: 是与Idle相对的一种状态。



3、SPI接口的数据通信

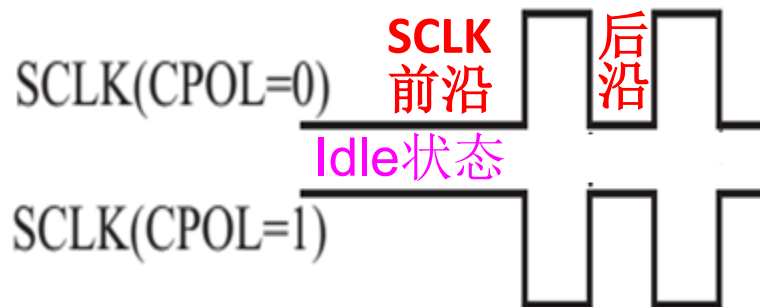
(6) 数据格式

时钟极性控制位**CPOL**用于设置时钟**SCLK**的极性。

CPOL=0: Idle状态=低电平, Active状态=高电平。

CPOL=1: Idle状态=高电平, Active状态=低电平。

- 从Idle状态到Active状态的转变, 称为**SCLK**前沿;
- 从Active状态到Idle状态的转变, 称为**SCLK**后沿。
- 一个**SCLK**前沿和后沿构成一个**SCLK**时钟周期, 一个**SCLK**时钟周期传输一位数据。



(6) 数据格式

SPI时钟相位选择控制位**CPHA**用于设置采样和改变数据的时钟**SCLK**的边沿:

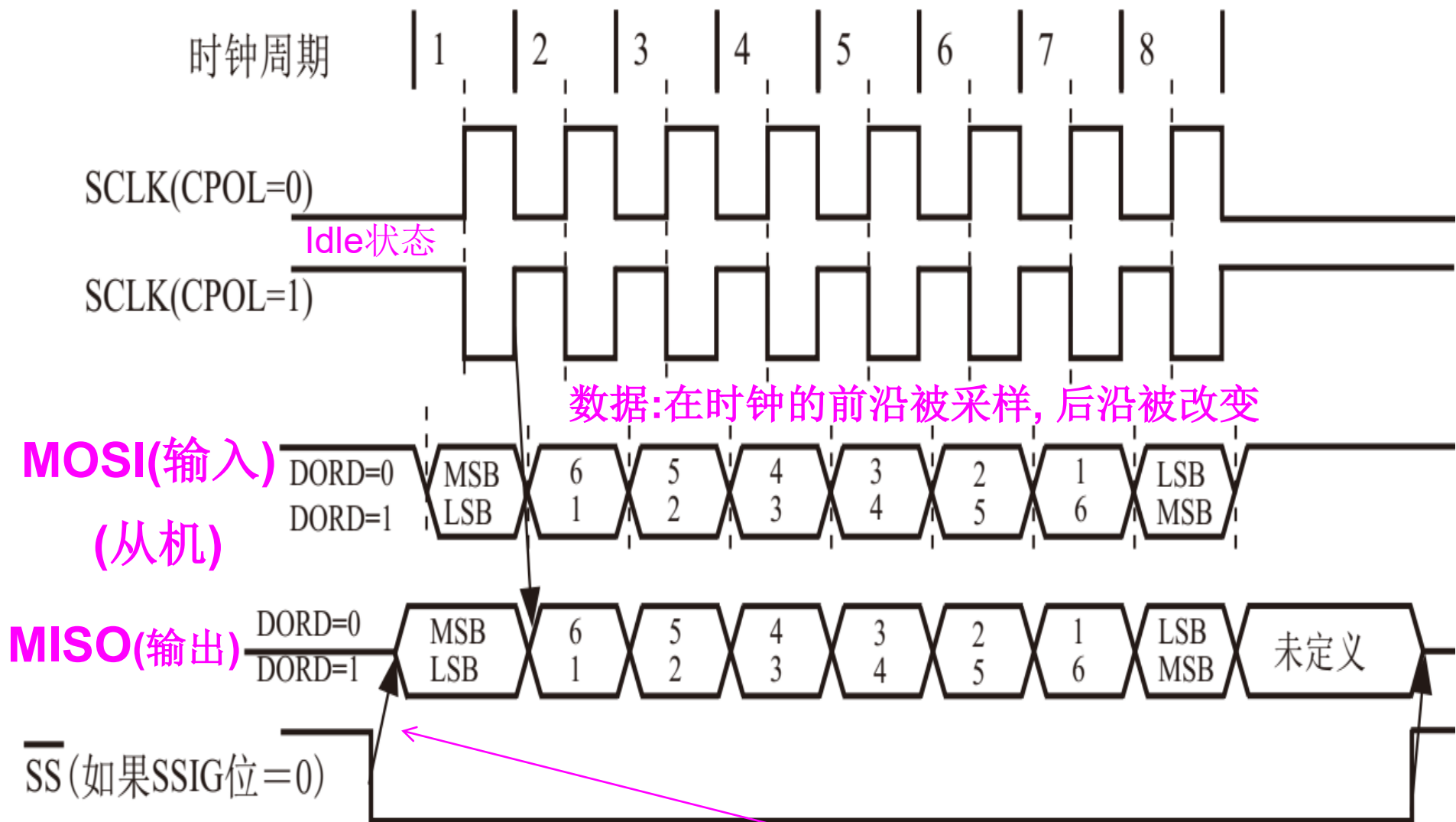
CPHA=1: 数据在时钟**SCLK**的**前沿驱动**到SPI口线, SPI模块在时钟**后沿采样**。

CPHA=0: 在时钟**SCLK**的**后沿被改变**, 并在时钟**前沿被采样**。
对从机,数据在/**SS**为低 (**SSIG=0**) 时驱动到SPI口线。
(注: **SSIG=1**时的操作未定义)

◇不同的**CPHA**, 主机和从机对应数据格式如**下页图**所示。

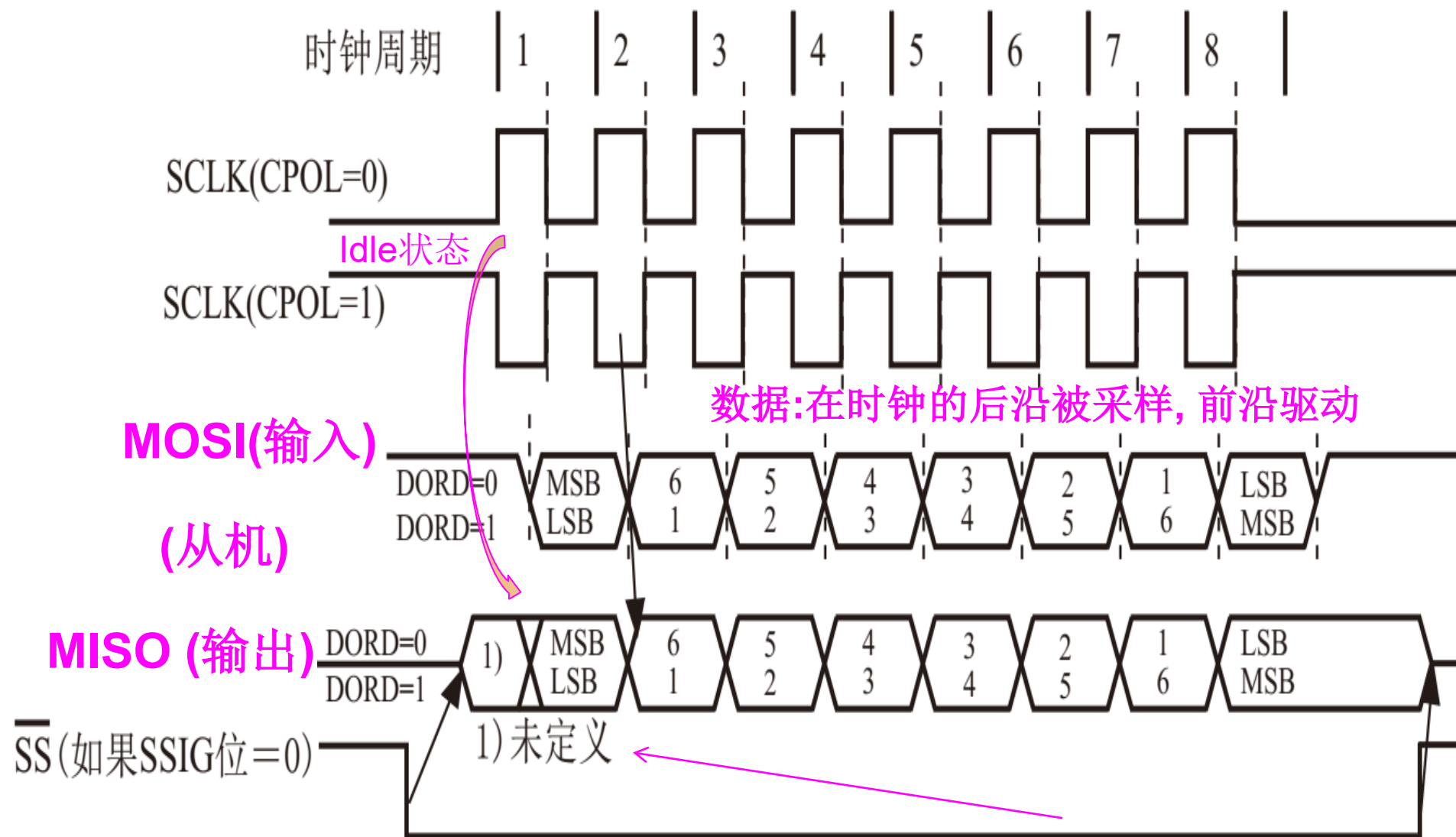
SPCTL	D7	D6	D5	D4	D3	D2	D1	D0
寄存器	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

(6) 数据格式



CPHA=0时SPI从机传输格式

(6) 数据格式



CPHA=1时SPI从机传输格式

3、SPI接口的数据通信 (7) SPI时钟预分频器选择

SPI时钟预分频器选择是通过SPCTL寄存器的SPR1, SPR0位实现的。

SPI控制寄存器SPCTL

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

SPI时钟频率的选择(STC15W系列)

SPR1	SPR0	时钟 (SCLK)
0	0	CPU_CLK/4
0	1	CPU_CLK/8
1	0	CPU_CLK/16
1	1	CPU_CLK/32

CPU_CLK是CPU时钟

4. SPI接口应用举例

(1) SPI相关的特殊功能寄存器

与SPI接口有关的特殊功能寄存器

寄存器	地址	D7	D6	D5	D4	D3	D2	D1	D0	复位值
SPCTL	CEH	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	00000 100B
SPSTAT	CDH	SPIF	WCOL	-	-	-	-	-	-	00xxx xxxB
SPDAT	CFH									00000 000B

4. SPI接口的应用举例

(1) SPI相关的特殊功能寄存器

1) SPI控制寄存器 (SPCTL)

SPCTL(地址为CEH,复位值为00H)各位定义如下:

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

①**SSIG**: 引脚忽略控制位。 (**SS** pin is **ignored** or not)

- 1: 由**MSTR**位确定器件为主机(**MSTR=1**)还是从机。
- 0: 由**/SS**引脚的低电平,决定从机被选中,即使主机也变为从机。**/SS**脚可作为I/O口使用, 可通过控制**/SS**脚的电平高低,使其被选中为从机与否。

(1) SPI相关的特殊功能寄存器

1) SPI控制寄存器 (SPCTL)

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

②**SPEN**: SPI使能位。

1: SPI使能。

0: SPI被禁止，所有SPI管脚都作为I/O口使用。

③**DORD**: 设定数据发送和接收的位顺序。

1: 数据字的最低位(LSB)最先传送；

0: 数据字的最高位(MSB)最先传送。

1) SPI控制寄存器 (SPCTL)

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

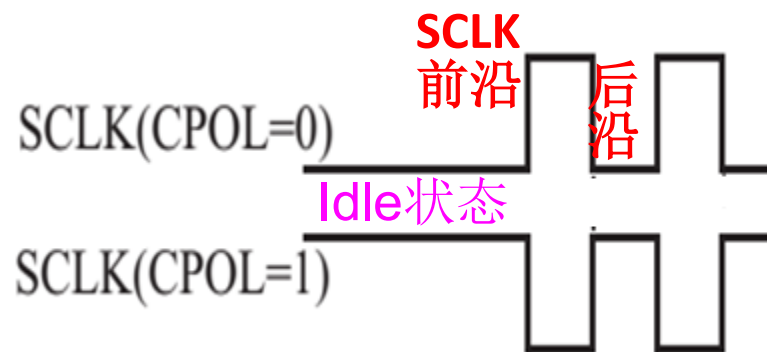
④ **MSTR**: SPI主/从模式选择位。

一般**MSTR=1**选择主机, **MSTR=0**选从机, 具体见[表8-8](#)。

⑤ **CPOL**: SPI时钟极性。

1: SPI空闲时**SCK=1**。SCLK的时钟前沿为下降沿而后沿为上升沿。

0: SPI空闲时**SCK=0**。SCLK的时钟前沿为上升沿而后沿为下降沿。



(1) SPI相关的特殊功能寄存器

1) SPI控制寄存器 (SPCTL)

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

⑥**CPHA**: SPI时钟相位选择控制。

1: 数据在SCLK的前时钟沿驱动到SPI口线，SPI模块在后时钟沿采样。

0: 在SCLK的后时钟沿被改变，并在前时钟沿被采样。数据在/SS为低 (SSIG=0) 时驱动到SPI口线。（注：SSIG=1时的操作未定义）

1) SPI控制寄存器 (SPCTL)

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0

⑦**SPR1**: 与SPR0联合构成SPI时钟速率选择控制位。

⑧**SPR0**: 与SPR1联合构成SPI时钟速率选择控制位。

SPI时钟频率的选择(STC15W系列)

SPR1	SPR0	时钟 (SCLK)
0	0	CPU_CLK/4
0	1	CPU_CLK/8
1	0	CPU_CLK/16
1	1	CPU_CLK/32

(STC15F/L系列)

SPR1	SPR0	时钟 (SCLK)
0	0	CPU_CLK/4
0	1	CPU_CLK/16
1	0	CPU_CLK/64
1	1	CPU_CLK/128

(1) SPI相关的特殊功能寄存器

2) SPI状态寄存器 (SPSTAT)

SPSTAT(地址: CDH, 复位值: 00XXXXXXB)各位定义:

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SPIF	WCOL	-	-	-	-	-	-

①SPIF: SPI传输完成标志。

- 当一次传输完成时, SPIF被置位。此时, 若SPI中断被打开(即ESPI (IE2.1)=1, EA (IE.7)=1), 将产生中断。
- 当SPI处于主模式且SSIG=0时, 如果 $\overline{SS}$ 为输入并被驱动为低电平, SPIF也将置位, 表示“模式改变”。
- SPIF标志通过软件向其写入"1" 而清0。

IE2	D7	D6	D5	D4	D3	D2	D1	D0
寄存器		ET4	ET3	ES4	ES3	ET2	ESPI	ES2

2) SPI状态寄存器 (SPSTAT)

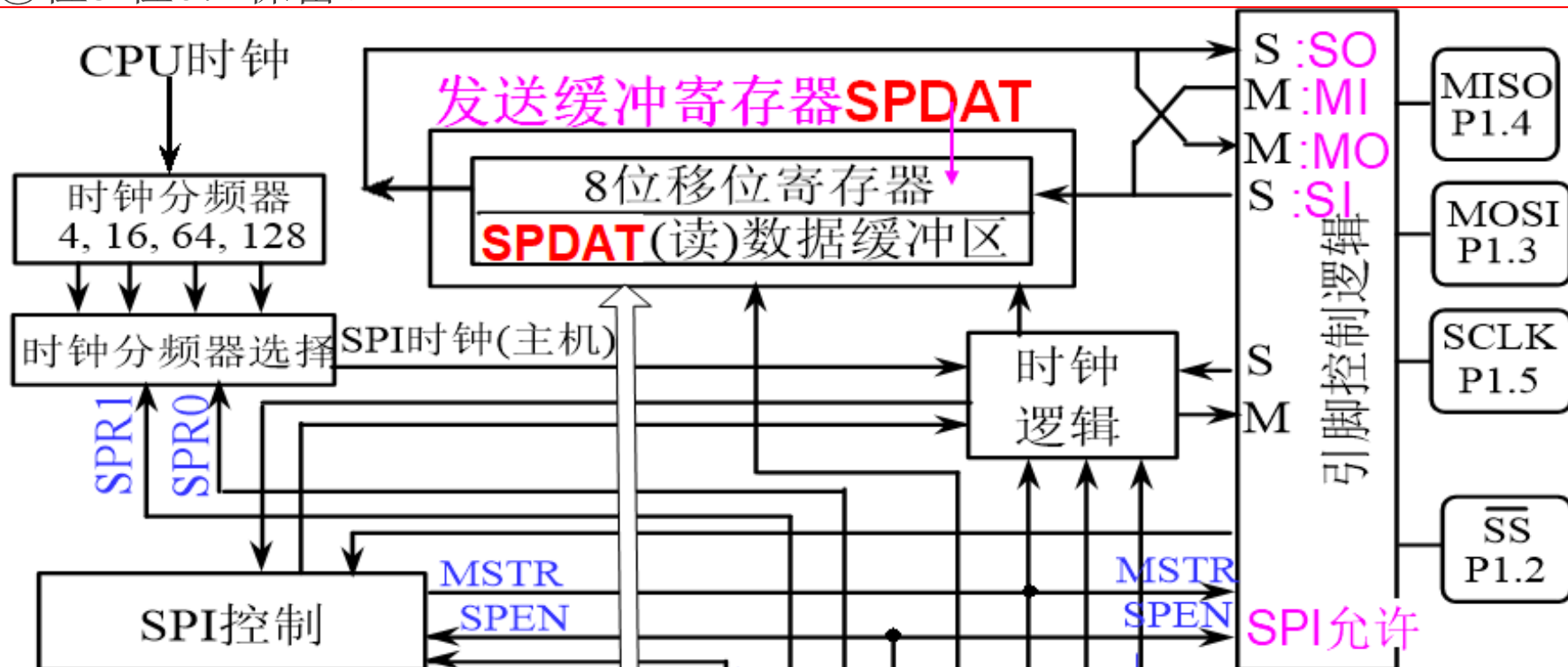
位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	SPIF	WCOL	-	-	-	-	-	-

② **WCOL**: SPI写冲突标志。

当一个数据还在传输时，又向数据寄存器**SPDAT**写入数据，WCOL将被置位。

WCOL 标志通过软件向其写入"1"而清0。

③ 位5-位0: 保留。

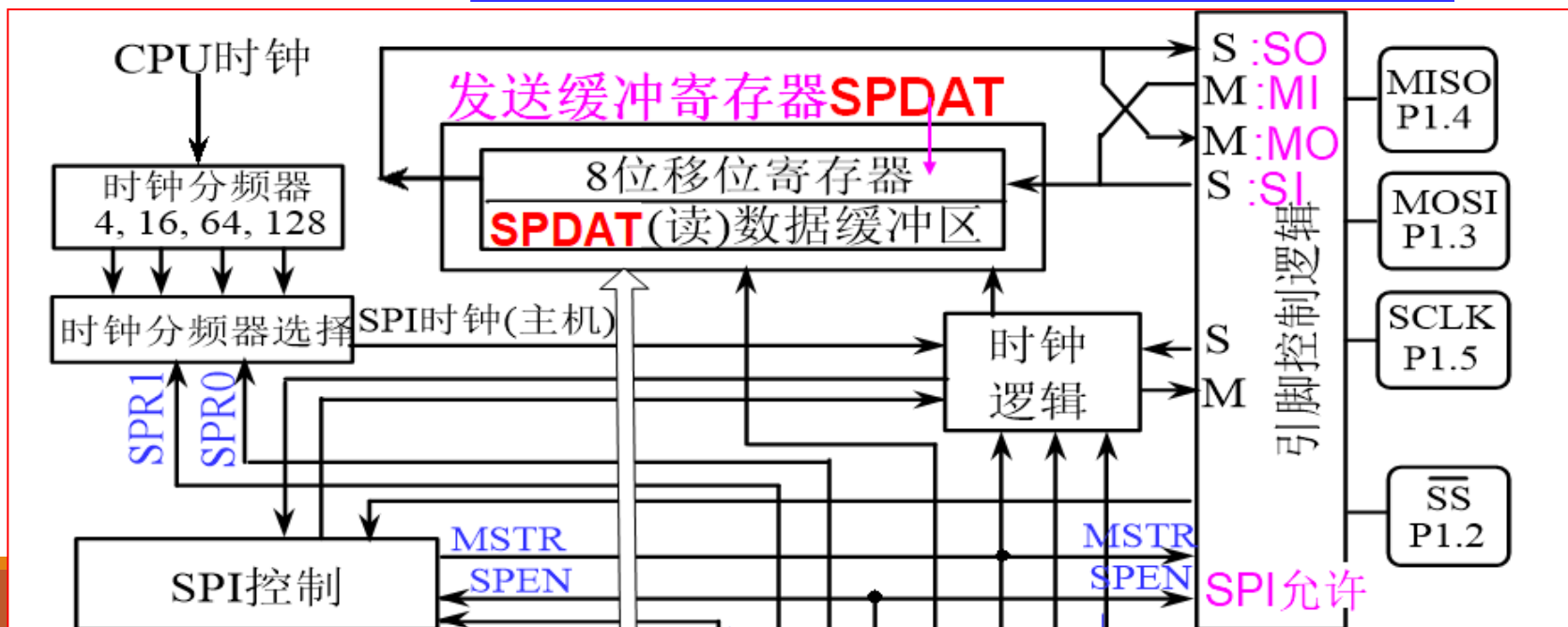


(1) SPI相关的特殊功能寄存器

SPDAT (地址为CFH, 复位值为00H)各位的定义如下:

位号	D7	D6	D5	D4	D3	D2	D1	D0
位名称	MSB							LSB

◇位7-位0: 保存**SPI通信数据字节**。其中, **MSB**为最高位, **LSB**为最低位。**spdata: 接收、发送同址共用。**



4. SPI接口应用举例

(2) 编程实例

SPI接口的使用包括SPI接口的初始化和SPI中断服务程序的编写。

SPI接口的初始化包括以下几个方面：

- ① 通过SPI控制寄存器SPCTL设置：/SS引脚的控制、SPI使能、数据传送的位顺序、设置为主机或从机、SPI时钟极性、SPI时钟相位、SPI时钟选择。

具体内容请参见SPI控制寄存器SPCTL介绍。

SPCTL寄存器	D7	D6	D5	D4	D3	D2	D1	D0
	SSIG	SPEN	DORD	MSTR	CPOL	CPHA	SPR1	SPR0
	1	1	0	1	1	1	1	0

SPI接口的初始化包括以下几个方面：

- ② 清0寄存器SPSTAT中的标志位SPIF和WCOL（向这两个标志位写1即可清零）。
- ③ 开放SPI中断（IE2中的ESPI=1，IE2寄存器不能位寻址，可以使用“或”指令：IE2 | =0x02）。
- ④ 开放总中断（IE中的EA=1）。

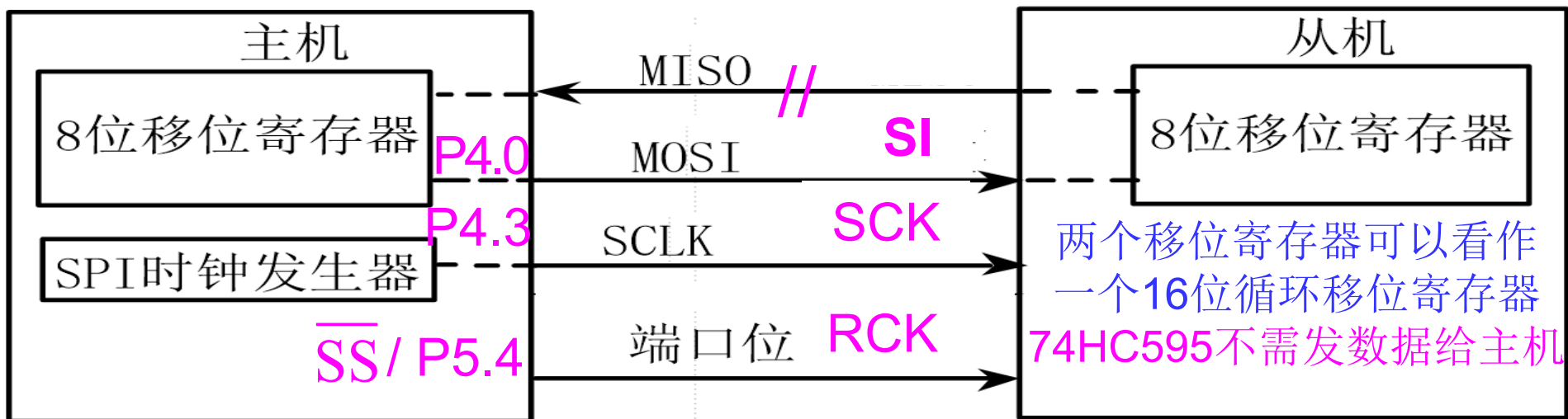
SPI中断服务程序根据实际需要进行编写。需要注意的是，在中断服务程序中首先需要将标志位SPIF和WCOL清0。因为SPI中断标志不会自动清除。

位号	D7	D6	D5	D4	D3	D2	D1	D0
IE2		ET4	ET3	ES4	ES3	ET2	ESPI	ES2
0x02:	0	0	0	0	0	0	1	0

74HC595接口及其应用



- 下面以单主机-单从机通信方式的应用为例说明SPI接口使用方法。用IAP15W4K58S4单片机向移位寄存器74HC595输出数据, 从而控制数码管LED显示。
- 74HC595是为Motorola的SPI总线开发的一款串行--并行转换芯片(8位3态移位寄存器/输出锁存器)。
- 由于74HC595的输入输出电平兼容TTL、NMOS和CMOS电平, 且具有较强的输出负载能力, 因此被广泛地运用于MCU(微控制器)和MPU(微处理器)的I/O口扩展。



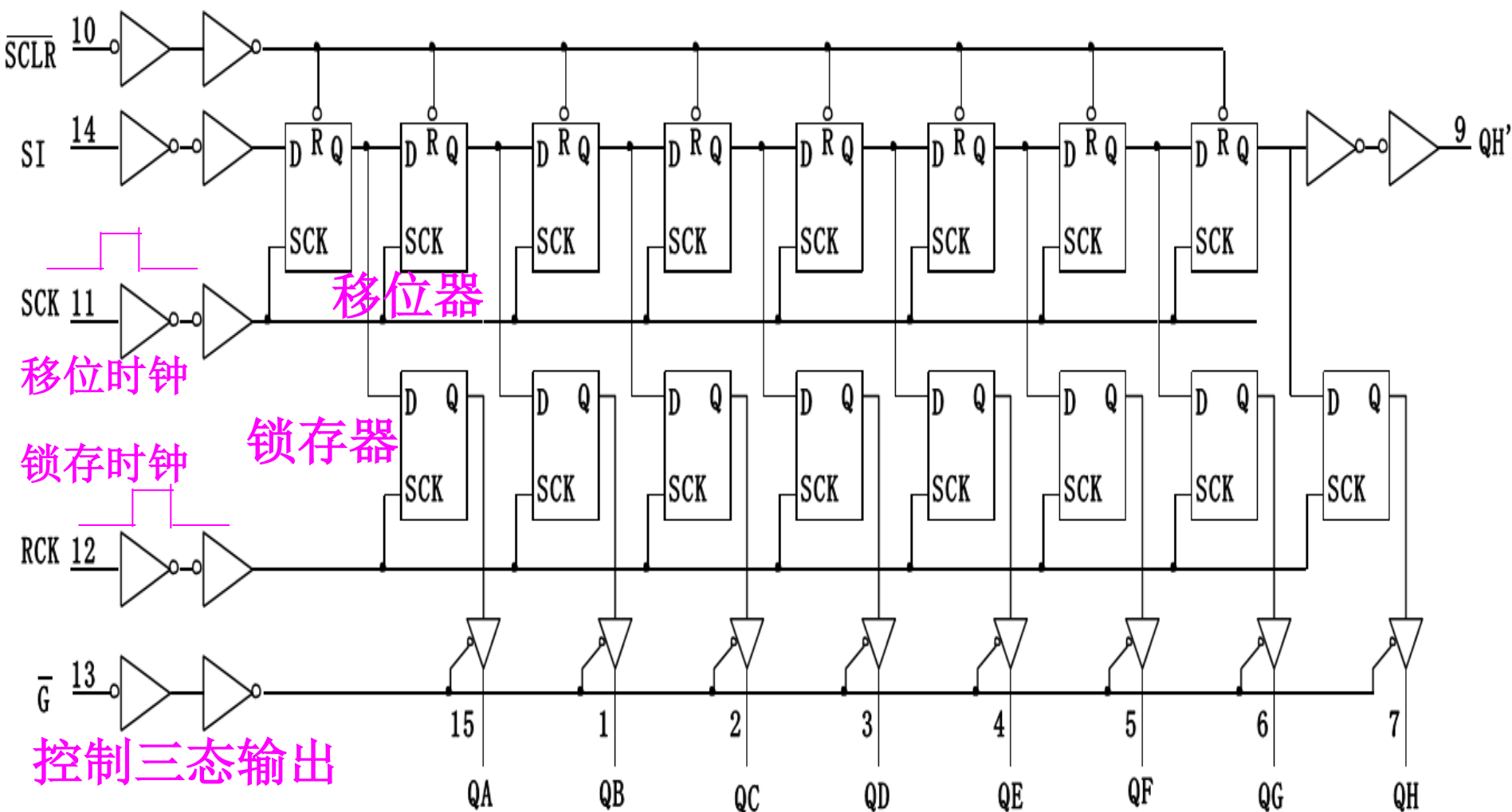
74HC595接口及其应用

74HC595在5V供电的时候能够达到30MHz的时钟速度，每个并行输出端口均能承受20mA的灌电流和拉电流。这个特点保证了不用增加额外的扩流电路即可轻松的驱动LED。

它的输入端允许500nS的上升（下降）时间，对严重畸形的时钟脉冲仍能检测。

这样就可以容纳较大的传输线对地电容，使系统的抗干扰能力增强。

74HC595接口及其应用



74HC595的逻辑电路图

74HC595接口及其应用

◆74HC595管脚功能描述如下:

◆SI: 串行数据输入, 数据从这个管脚移进内部的8位串行移位寄存器。

◦ SCK: 移位寄存器时钟输入: 上升沿, 数据寄存器数据移位: QA→QB→QC→...→QH; 下降沿寄存器数据不变。

(脉冲宽度: 5V时, 大于几十纳秒即可.)

◦ QH': 串行数据输出, 用于级联。无三态输出功能。

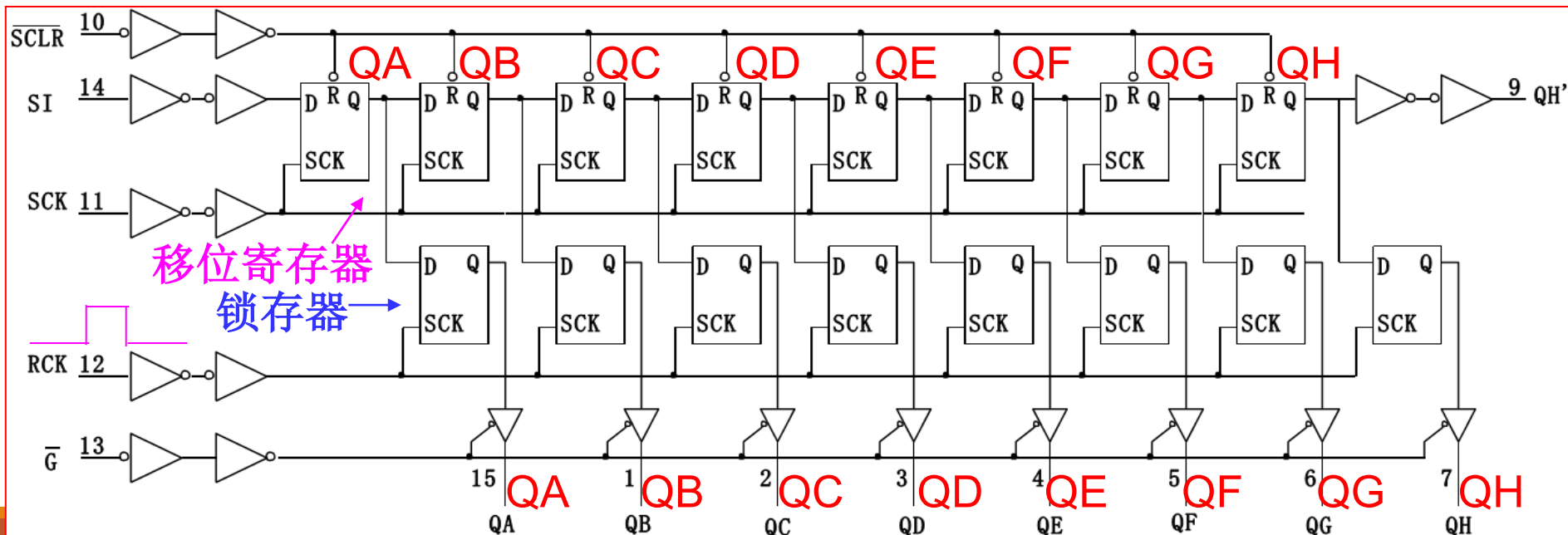
◦ QA~QH: 锁存器并行数据输出, 三态

◦ /G: 输出使能控制端。低电平有效, 将锁存器的输出映射到输出并行口(QA-QH)上。当输入高电平时, 高阻态。

74HC595			
10	/SCLR	QA	15
16	VCC	QB	1
11	SCK	QC	2
14	SI	QD	3
12	RCK	QE	4
13	/G	QF	5
9	QH'	QG	6
8	GND	QH	7

74HC595接口及其应用

- **RCK**: 存储寄存器(锁存器)时钟输入, 上升沿时, 移位寄存器的数据进入数据存储寄存器(锁存器), 下降沿时存储寄存器数据不变, 通常将**RCK**置为低电平。
- 当移位结束后, 在**RCK**端产生一个正脉冲(5V供电时,大于几十纳秒即可), 更新输出数据。
- **/SCLR**: 移位寄存器清零输入。低电平有效, 当此管脚上出现低电平时, 将复位内部的移位寄存器(清0), 但不影响8位锁存器的值。通常工作时可接**Vcc**。
- **GND**: 电源地
- **VCC**: 电源正, 一般接5VDC。



2.5 I²C通信接口

1、I²C总线简介

I²C（Inter-Integrated Circuit）总线是由PHILIPS公司开发的串行总线，用于连接微控制器及其外围设备。

I²C总线产生于二十世纪80年代，最初为音频和视频设备开发，如今主要在服务器管理中使用，其中包括单个组件状态的通信。

例如，管理员可对各个组件进行查询，以管理系统的配置或掌握组件的功能状态，如电源和系统风扇。可随时监控内存、硬盘、网络、系统温度等多个参数，增加了系统的安全性，方便了管理。